

药材烘干过程的温度与水分浓度场演化模型

摘要

烘干是决定中药材品质的关键工序。本文以圆柱药材为对象，建立了热风条件下内部温度与干基水分浓度耦合输运的一维径向机理模型，为四个子问题给出了一致的求解框架与定量结果。

针对问题一（附录 2 常数物性、0–1800 s 预热平衡），将傅里叶导热与变系数 Fick 扩散方程经 Robin 对流边界与附件 1 风况调度联立，采用节点控制体有限体积–Crank–Nicolson（物性中点 Picard 迭代）格式求解， $\Delta r = 0.1\text{ cm}$ 、 $\Delta t = 0.25\text{ s}$ ，输出表 1、表 2 及 1 s 分辨率的 result1.xlsx：1800 s 时中心升温 5.58 °C、表面水分已降至 1.5116 kg/kg，呈现“表层先行失水、芯部滞后响应”的格局。

针对问题二，物性改用附录 3 经验式并衔接恒温段（50.013 °C、0.04999 kg/kg），得到表 3、表 4：3 h 末中心温度已达风温的 99.6%，而中心水分仅降至 2.1605 kg/kg，热、质过程时间尺度分离显著。针对问题三，以“全场 $C < 0.15$ ”为判停条件，得烘干时长 $t_{\text{end}} = 59273\text{ s} \approx 16.4647\text{ h}$ （表 5、result3.xlsx）。

针对问题四，引入随体坐标将附件 2 给出的半径收缩 $R(t)$ （2.000→1.198 cm）映射为固定域问题，配合附录 4 物性求得 $t_{\text{end}} = 70758\text{ s} \approx 19.6550\text{ h}$ ，判停时半径 1.2107 cm（表 6、result4.xlsx）；相对“附录 4 物性、不收缩”的对照基准 39.20 h，收缩使时长缩短 49.87%。

灵敏度分析表明：烘干时长由传质链控制—— h_m 、 D 的 $\pm 20\%$ 扰动引起 $-11.1\% \sim +16.8\%$ （问题三）与 $-9.1\% \sim +13.5\%$ （问题四）的时长变化，而换热与导热参数影响不足 0.14%；判停结果对网格/步长（ $\leq 0.01\%$ ）、恒温取值（0.16%）、端面修正（4.99%）与潜热耦合（0.02%）均稳健。据此建议优先以强化表面换质（提风速、降排潮湿度）调控干燥制度。

关键词： 传热传质 有限体积法 Crank–Nicolson 格式 判停条件 动边界收缩
灵敏度分析

一、问题重述

1.1 问题背景

干燥是决定中药材成品品质的关键工序之一。热风烘干通过调控烘房的温度与湿度环境完成药材脱水,工艺一般分为预热平衡与恒温干燥两个阶段:预热平衡阶段以升温、水分重新分布为主;恒温干燥阶段在稳定的热风环境下持续脱除水分。工艺参数选取不当会造成干燥效率低、能耗高、成品品质不稳定,而单纯依靠试验优化成本高、周期长^[5],需要借助数理分析与数值仿真获得药材内部温度与水分浓度的演化规律,进而确定烘干时长等关键工艺量。

1.2 问题内容

药材近似为长 25 cm、半径 2 cm 的圆柱体,其内温度与水分浓度(干基含水率,单位 kg/kg)随时间、径向位置变化;烘房空气的温度与水分浓度按附件数据给定。需要研究以下四个问题。

问题一: 已知初始温度 28 °C、初始水分浓度 2.55 kg/kg,烘房温度与水分浓度随时间的变化由附件 1 给出(间隔 60 s)。物性参数按附录 2 取常数,水分扩散系数满足经验式 $D = 7 \times 10^{-9} e^{-\frac{0.89}{C}}$ 。建立预热平衡阶段(0–1800 s)药材内部温度场与水分浓度场的数学模型,按表 1、表 2 格式给出若干典型时刻、不同半径处的结果,并将 1 s、0.1 cm 分辨率的完整结果写入 result1.xlsx。

问题二: 实际烘干一般持续 2–3 天,预热平衡与恒温干燥两阶段的参数不同。为简化问题,相关物性经验公式统一采用附录 3,建立整个烘干过程药材温度与水分浓度变化的数学模型;按表 3、表 4 格式给出 3 h 内每隔 0.5 h 的结果,并以 1 s、0.1 cm 分辨率写入 result2.xlsx。

问题三: 烘干要求药材各处水分浓度低于 0.15 kg/kg,确定完成干燥所需的烘干时长;按表 5 格式给出每隔 6 h 的结果,并以 60 s、0.1 cm 分辨率写入 result3.xlsx。

问题四: 实际烘干中药材因失水发生尺寸收缩,半径随时间变化由附件 2 给出;物性经验公式改用附录 4。确定考虑收缩后的烘干时长;按表 6 格式给出每隔 6 h 的结果(距离列到当前表面为止),以 60 s、0.1 cm 分辨率写入 result4.xlsx。

所有结果保留四位小数。

二、问题分析

2.1 题目特点与建模要素

本题是典型的传热—传质耦合的机理建模(动力学类)问题:圆柱药材内部的温度 $T(r, t)$ 与干基水分浓度 $C(r, t)$ 由扩散型偏微分方程控制,边界上通过牛顿冷却定律与对流传质定律^[2]与烘房空气耦合;物性(密度、比热容、导热系数、扩散系数)随含水率(以及问题 2–4 中随温度)按题给经验公式变化,使方程组呈非线性耦合。药材长 25 cm、半径 2 cm,长半径比达 12.5:1,且题面要求的空间位置均为“到药材中心的距离”(0–2 cm),故可将问题简化为一维径向输运;两个端面的影响通过面积修正系数在灵敏度分析中单独检验。

判断模型尺度的关键无量纲数为毕渥数:预热阶段 $Bi_h = h \frac{R}{k} = 25 \times \frac{0.02}{0.36} \approx 1.39$, $Bi_m = h_m \frac{R}{D} \approx 3.2$ 。两者均远大于 0.1,说明内部导热/扩散阻力与表面换热/换质阻力同

量级，集总参数法不可用，必须求解带 Robin 边界的分布参数模型——这也与题面要求给出多个半径处的场值相一致。

2.2 各子问题的分析

问题一：物性为附录 2 常数、扩散系数依赖局部含水率。核心是建立圆柱坐标下二维场（时间—半径）的初边值问题，并以高精度数值解输出 7 个时刻、5 个半径的表格以及 1 s/0.1 cm 全分辨率结果。30 min 内风温尚在爬升（28→41.5 °C），需按附件 1 数据对边界条件作时间插值。

问题二：从“预热平衡”推广到“全过程”：物性改用附录 3 的统一经验公式（显式依赖 C ，扩散系数同时依赖 T ）；烘房条件在附件 1 数据段（0–14400 s，预热平衡阶段）后进入恒温干燥阶段——风温稳定在 50 °C 附近、湿度稳定在 0.05 kg/kg 附近，此后按数据末段稳定值恒定延拓。论文只需 3 h 结果（仍处于附件 1 覆盖段），但模型必须自洽地支持问题 3 的长时外推。

问题三：是问题 2 模型的外推与判停问题：以“各处 $C < 0.15$ ”为完成标志（由于水分场单调内移，中心处 $C(0, t)$ 恒为最大值，该判据等价于 $C(0, t) = 0.15$ 的最早到达时刻），需要长时间积分并对事件时刻做细分定位；同时给出结束时刻行的全场剖面。

问题四：在问题 3 框架上增加几何非线性：半径 $R(t)$ 按附件 2 从 2.000 cm 单调收缩至 1.198 cm。物理上材料随脱水收缩、各材料点相对位置比例不变，因此采用随体坐标 $\xi = \frac{r}{R(t)}$ 将动边界化为固定域，物性改用附录 4。求解后同样按 0.15 判停得到收缩情形下的烘干时长。

2.3 总体思路

先统一建立“一维径向热–质耦合 Robin 初边值问题”的机理模型，再按子问题替换物性参数集、边界调度与判停条件；数值上采用节点控制体有限体积法（保证离散守恒）与 Crank–Nicolson 时间推进（物性中点 Picard 迭代处理非线性）；结果经守恒闭合、网格/步长收敛与单调性三重校验后，输出题面规定的六张表与四个结果文件；最后围绕传质参数（ h_m 、 D ）的主导性开展灵敏度分析，并对端面修正、恒温段取值、蒸发潜热耦合等模型假设进行定量检验。整体技术路线如图 1 所示。

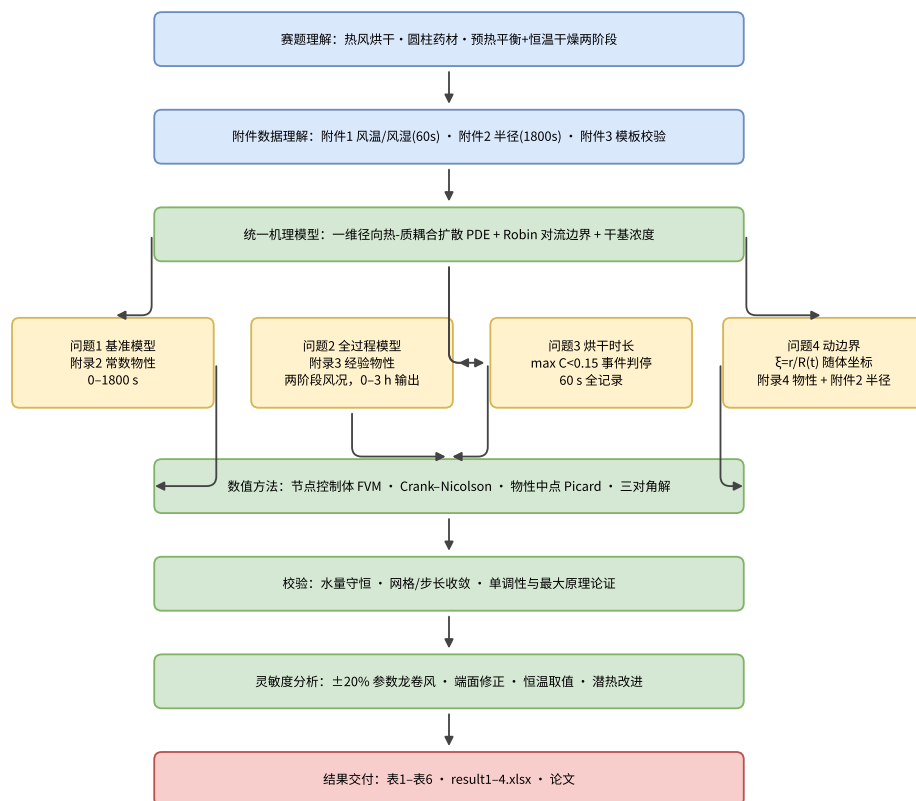


图 1 求解本赛题的整体技术路线图

数据读取、校验到求解、输出的处理链条如图 2 所示：附件数据先经断言检查（步长、单调性），再构造边界调度器与物性装配；求解结果直接生成论文表与 result1-4.xlsx，校验不通过则返回修正。

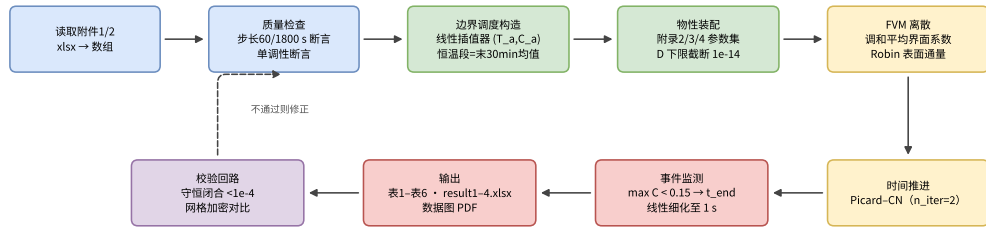


图 2 数据处理与数值求解总体流程

三、模型假设

结合题给条件与工程背景，本文作如下假设。

1. 假设一：药材材质均匀、各向同性，物性仅依赖局部水分浓度与温度（按题给经验公式），不含化学反应热源与生质。**理由**：题面仅给出随 C 、 T 变化的经验物性，无组分信息；药材烘干温度不高（ $\leq 50^\circ\text{C}$ ），化学变化可忽略。**影响**：若材质不均，仅个别位置剖面出现扰动，判停时刻变化有限。
2. 假设二：忽略轴向梯度，将圆柱药材视为无限长轴对称问题，仅保留径向一维输运；端面影响以面积修正系数 $1 + \frac{R}{L}$ 检验。**理由**： $\frac{L}{R} = 12.5$ ，轴向扩散时间尺度 $\left(\frac{L}{2}\right)^2 \frac{1}{\alpha} \approx 25.7$ h，远大于径向 $\frac{R^2}{\alpha} \approx 0.66$ h（ $\alpha = \frac{k}{\rho c_p} = 1.69 \times 10^{-7} \frac{\text{m}^2}{\text{s}}$ ）；且题目要求的所有位置均按“到中心的距离”给出。**检验**：灵敏度分析第 9.3 节表明端面修正使烘干时长仅缩短 4.99%，不改变结论。
3. 假设三：固相骨架连续、无裂纹与毛细对流，水分迁移遵循 Fick 扩散定律，扩散系数即题给 $D(C, T)$ ；表面向空气的移质速率由对流传质系数 h_m 与表面—空气浓度差线性驱动。**理由**：这是题给参数体系的封闭口径（ D 、 h_m 单位相容）；无平衡吸湿等温线数据，故不引入吸附模型。
4. 假设四：对流换热边界为牛顿冷却定律，忽略辐射与蒸发潜热对温度场的耦合（蒸发热汇量级见假设八）。**检验**：第 9.4 节加入潜热项后烘干时长仅变化 +0.02%。
5. 假设五：预热平衡阶段的烘房条件由附件 1 线性插值给出（数据间认为变化线性）；14400 s 后进入恒温干燥阶段，风温、风湿度分别恒定于附件 1 末 30 min 均值 50.013°C 与 0.04999 kg/kg 。**理由**：附件 1 数据在末段已稳定（ $\sigma_T = 0.15^\circ\text{C}$ ），均值化滤除控温噪声；恒温取值方式的差异仅引起 -0.16% 的时长变化（第 9.4 节）。
6. 假设六：初态温度与含水率在截面内均匀（ 28°C 、 2.55 kg/kg ）。**理由**：题面给定“开始时”单值条件。
7. 假设七（问题四）：药材均匀径向收缩，材料点的归一化半径 $\xi = \frac{r}{R(t)}$ 不随时间变化；收缩过程中忽略骨架应变能、体积功与密度经验式对质量守恒的偏离。**理由**：附件 2 仅给出半径时程，均匀收缩是与数据相容的最简描述；题给 $\rho(C)$ 为经验体积密度公式，本身未内蕴干物质守恒，故方程按点值输运形式建立。

8. 假设八：问题 2、3 中两阶段的对流换热/传质系数沿用附录 2 口径 ($h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$, $h_m = 8 \times 10^{-7} \frac{\text{m}}{\text{s}}$)。**理由：**题面未为恒温阶段另给系数；两参数恰是第 9.2 节灵敏度考察的对象，其主导性已定量给出。

四、符号说明

本文主要符号及含义如下表所示，其余符号在首次出现处说明。

符号	含义	单位
r, R, L	径向坐标；药材当前半径；药材长度	m (cm)
$\xi = \frac{r}{R(t)}$	随体归一化径向坐标（问题四）	—
t, t_{end}	时间；烘干结束时刻	s；h
$T(r, t)$	药材温度	°C
$T_{a(t)}$	烘房风温	°C
$C(r, t)$	药材干基水分浓度	kg/kg
$C_{a(t)}$	烘房空气水分浓度	kg/kg
$\rho(C)$	密度	kg/m ³
$c_p(C)$	比热容	J/(kg·K)
$k(C)$	热传导系数	W/(m·K)
$D(C, T)$	水分浓度扩散系数	m ² /s
h	对流换热系数	W/(m ² ·K)
h_m	对流传质系数	m/s
$\xi_j = \frac{j}{N}$	径向节点 ($j = 0, \dots, N$)	—
$\Delta r, \Delta t$	径向/时间步长	m；s
Bi_h, Bi_m	热/质毕渥数 $h \frac{R}{k}$ 、 $h_m \frac{R}{D}$	—
$\text{Fo} = D \frac{t}{R^2}$	傅里叶数（扩散时间尺度）	—

五、问题一的模型建立与求解

5.1 控制方程的建立

设 $t = 0$ 时药材温度 $T_0 = 28 \text{ °C}$ 、干基水分浓度 $C_0 = 2.55 \text{ kg/kg}$ ；圆柱内只沿径向变化，任取微元 $[r, r + \Delta r]$ （取单位长度）。

能量平衡。单位长度微元热容量为 $\rho c_p \cdot 2\pi r \Delta r$ ，Fourier 导热在两个柱面上的净输入为 $[(\frac{\partial}{\partial r})(kr \frac{\partial T}{\partial r})] \Delta r$ ，得

$$\rho c_p \frac{\partial T}{\partial t} = k \left[\frac{\partial^2 T}{\partial r^2} + \frac{1}{r} \frac{\partial T}{\partial r} \right], \quad 0 < r < R, t > 0. \quad (1)$$

水分平衡。干基浓度以绝干物料质量为度量基准，固相骨架不动时水分守恒直接给出 Fick 型扩散方程

$$\frac{\partial C}{\partial t} = \frac{1}{r} \left(\frac{\partial}{\partial r} \right) \left[r D(C) \frac{\partial C}{\partial r} \right], \quad (2)$$

其中扩散系数采用题给经验式

$$D(C) = 7 \times 10^{-9} e^{-\frac{0.89}{C}} \text{ m}^2/\text{s}. \quad (3)$$

边界与初始条件。轴对称性给出中心零通量；表面处导热/扩散通量与对流换热量、对流传质量平衡（牛顿冷却与线性移质驱动）：

$$\frac{\partial T}{\partial r} \Big|_{r=0} = \frac{\partial C}{\partial r} \Big|_{r=0} = 0, \quad (4)$$

$$k \frac{\partial T}{\partial r} \Big|_{r=R} = h [T_{a(t)} - T(R, t)], \quad D \frac{\partial C}{\partial r} \Big|_{r=R} = -h_m [C(R, t) - C_{a(t)}]. \quad (5)$$

$$T(r, 0) = 28^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg}. \quad (6)$$

式(5)右端烘房条件 $T_{a(t)}$ 、 $C_{a(t)}$ 由附件 1 的 241 组数据 (0–14400 s, 间隔 60 s) 作线性插值获得。参数取附录 2: $\rho = 820$ 、 $c_p = 2600$ 、 $k = 0.36$ 、 $h = 25$ 、 $h_m = 8 \times 10^{-7}$ 。式(1)–(6)构成封闭的初边值问题；两方程仅通过 $D(C)$ 与表面通量弱耦合，温度场与水分场可联立推进。

5.2 数值求解方法

采用节点控制体有限体积法。将 $[0, R]$ 均分为 $N = 20$ 个网格，节点 $r_j = j\Delta r$ ($\Delta r = 0.1 \text{ cm}$ ，使输出网格与计算网格重合)，节点控制体体积 $V_j = \pi(r_j^2 - r_{j-\frac{1}{2}}^2)$ 等按柱面积加权。界面扩散/导热系数取相邻节点值的调和平均，以保证 $D(C)$ 强变化时的通量连续性。记界面传导率

$$g_{j+\frac{1}{2}} = 2\pi r_{j+\frac{1}{2}} \frac{D_{j+\frac{1}{2}}}{\Delta}. \quad (7)$$

则控制体离散式为（以水分为例，温度同理并含 ρc_p ）

$$V_j \frac{C_j^{n+1} - C_j^n}{\Delta t} = \frac{1}{2} (F_j^n + F_j^{n+1}), \quad (8)$$

其中 F_j 为净界面通量与表面 Robin 通量之和。右端为 Crank–Nicolson 平均，对线性问题无条件稳定、二阶精度；对非线性物性，在每一步内将系数取中点场 $\frac{T^n + T^{n+1}}{2}$ 并作 2 轮 Picard 迭代，且新旧两端使用同一组系数，从而保证离散水量守恒精确成立并消除伪源项。

时间步长 $\Delta t = 0.25 \text{ s}$ ，逐步监测判据与守恒量；三对角方程组用追赶法 (Thomas) 求解，复杂度 $O(N\Delta t^{-1})$ ；离散与迭代格式的设计遵循文献^[1]的守恒性与有界性原则，变系数扩散问题的中点线性化亦见文献^[3]。该格式以单一求解器函数实现（见附录 A 的 simulate 例程），为问题一至四共用。

5.3 求解结果与检验

表 1 30 分钟内药材的温度 (单位: °C, 行=时刻, 列=到中心距离)

时间/s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	28.0001	28.0004	28.0041	28.0326	28.1786
300	28.0411	28.0638	28.1514	28.3676	28.8469
600	28.4537	28.5363	28.8039	29.3153	30.1640
900	29.3245	29.4584	29.8753	30.6156	31.7297
1200	30.5428	30.7098	31.2222	32.1121	33.4269
1500	31.9957	32.1867	32.7658	33.7460	35.1204
1800	33.5752	33.7719	34.3640	35.3619	36.7853

表 2 30 分钟内药材的水分浓度 (单位: kg/kg, 行=时刻, 列=到中心距离)

时间/s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	2.5500	2.5500	2.5500	2.5500	2.2880
300	2.5500	2.5500	2.5500	2.5486	2.0660
600	2.5500	2.5500	2.5500	2.5342	1.8844
900	2.5500	2.5500	2.5495	2.5041	1.7593
1200	2.5500	2.5500	2.5478	2.4648	1.6616
1500	2.5500	2.5499	2.5440	2.4212	1.5808
1800	2.5500	2.5496	2.5376	2.3764	1.5116

表 1、表 2 的 1 s、0.1 cm 全分辨率结果已按要求写入 result1.xlsx (“温度”“水分浓度”两个工作表, A 列为时间, 首行为距离)。

温度场的演化呈典型 Robin 边界升温过程 (图 3 左图): 热扰动自表面向内传播, 同一时刻温度随 r 单调上升; 1800 s 时中心升温 5.58 °C, 而该时刻风温已升至 41.513 °C, 表面温度 36.785 °C——表面与风温差 4.73 °C 正比于对流/传导热阻之比 Bi_h^{-1} 量级, 说明升温由表面换热与内部导热串联控制。水分场 (图 3 右图) 呈“表面迅速失水、芯部滞后”的形态: 2 cm 处 1800 s 已降至 1.5116, 中心仅 2.5497。其机理可由时间尺度解释: 水分自表面移出的特征时间 $\Delta \frac{r}{2h_m} \approx 625$ s 远小于扩散穿透时间 $\frac{R^2}{D} \approx 80$ min ($D \approx 5 \times 10^{-9}$ 于 $C = 2.55$), 前期表面半控制体先排干、内部梯度尚未建立。图 4 给出中心/表面两条时间曲线, 并对照风温与整体水量。

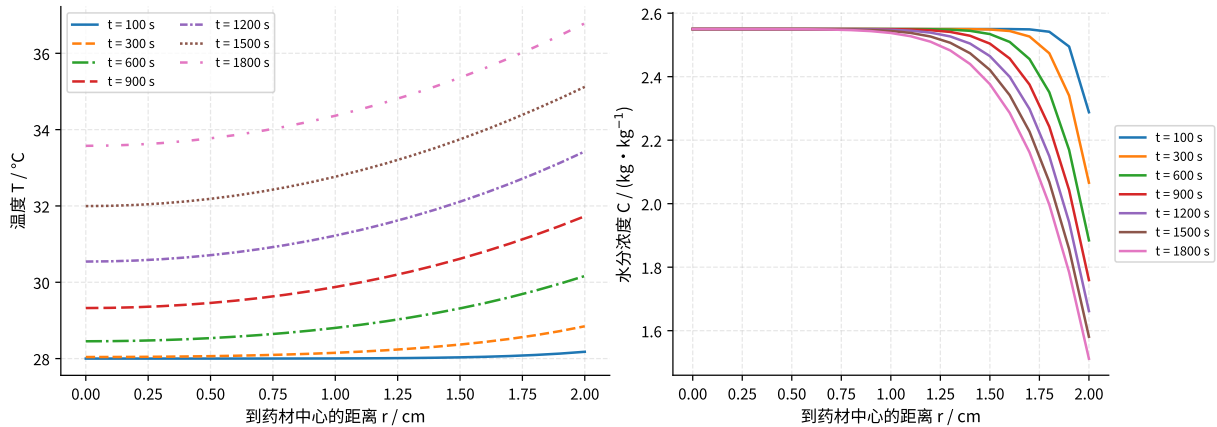


图 3 问题一: 不同时刻温度剖面 (左) 与水分浓度剖面 (右)

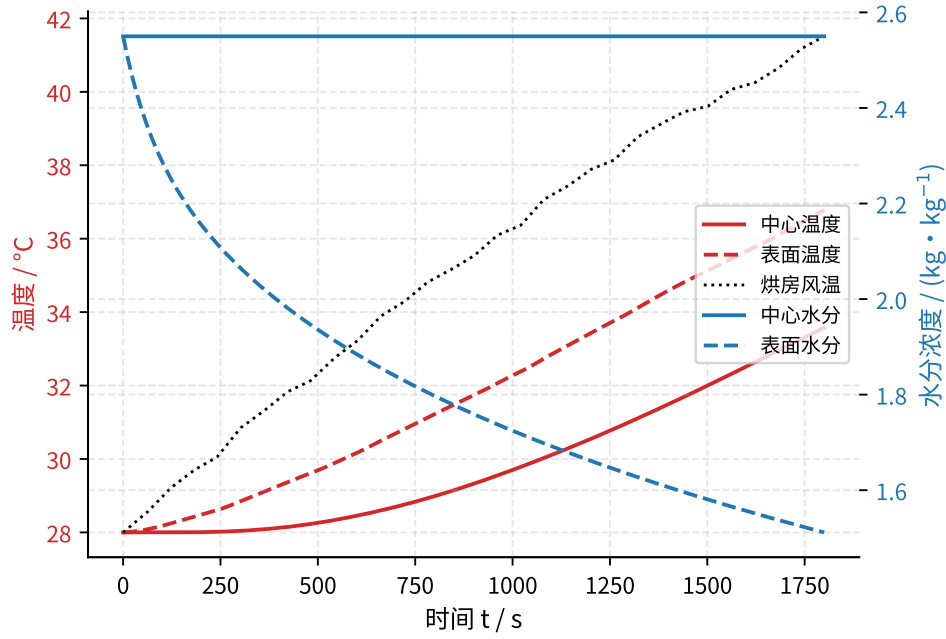


图 4 问题一：中心与表面的温度/水分浓度随时间演化（预热阶段）

上述剖面族与演化曲线分别如图 3 与图 4 所示，二者共同支持“预热平衡阶段以升温为主、失水集中于表层”的工艺判断。

数值检验。（1）守恒性：离散水量变化与表面移质积分闭合，相对误差 4.0×10^{-5} ；（2）网格收敛： $N = 20/40/80$ （步长同步减半）下，温度表值最大偏差 0.0015°C ，水分表值除早期（100 s）表面单元存在 0.0329 的 Robin 边界层离散差异外，300 s 后均 ≤ 0.006 ，80 节点结果与 40 节点差 0.0061（见第 9 节 第 9.2 小节与图 13）；（3）单调性：各点温度单调上升且不超风温，水分单调下降。

六、问题二的模型建立与求解

6.1 模型的推广

问题二要求覆盖预热平衡与恒温干燥的完整过程，物性不再取常数而统一采用附录 3 经验公式：

$$\rho(C) = 650 + 128C, \quad c_{p(C)} = 1450 + 2736 \frac{C}{C+1}, \quad (9)$$

$$k(C) = 0.21 + 0.38 \frac{C}{C+1}, \quad D(C, T) = 2.4 \times 10^{-3} e^{-0.45C} e^{-\frac{3850}{T+273.15}}. \quad (10)$$

控制方程仍为式(1)、式(2)的守恒形式，只是 ρ, c_p, k, D 成为 (r, t) 的局部函数：能量方程右端改用 $\frac{1}{r} \left(\frac{\partial}{\partial r} \right) [rk(C) \frac{\partial T}{\partial r}]$ ，比热容按体积热容 $\rho(C)c_{p(C)}$ 装配。为保持第 5 节的有限体积格式， k, D 在界面取调和平均， ρc_p 取单元内平均值；时间推进仍为物性中点 Picard 迭代的 Crank–Nicolson 格式，每步 2 轮迭代即可使系数–场值耦合收敛到 10^{-9} 量级。

烘房条件按假设五分两段： $t \leq 14400$ s 取附件 1 线性插值；其后恒温干燥阶段取 $T_a = 50.013^\circ\text{C}$ 、 $C_a = 0.04999$ kg/kg。附件 1 内恒温段（12600–14400 s）风温标准差仅 0.15°C ，验证了“恒定延拓”的合理性（见图 5）。

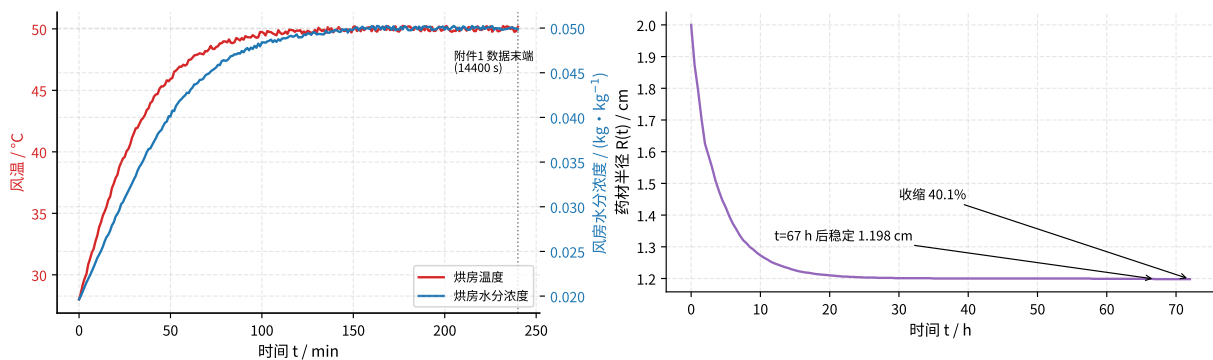


图 5 附件 1 烘房风温/风湿度调度 (左) 与附件 2 半径收缩时程 (右)

6.2 求解与结果

空间仍取 $N = 20$ ($\Delta r = 0.1$ cm)，积分至 $t = 3$ h ($\Delta t = 0.25$ s)。表 3、表 4 按题面格式给出 (行=到中心距离, 列=时刻)。

表 3 3 h 内药材的温度 (单位: °C)

距离	0.5 h	1 h	1.5 h	2 h	2.5 h	3 h
0 cm	32.1802	40.3276	45.7682	48.3874	49.4315	49.8328
0.5 cm	32.3724	40.4992	45.8582	48.4274	49.4450	49.8391
1 cm	32.9539	41.0060	46.1228	48.5443	49.4831	49.8599
1.5 cm	33.9446	41.8288	46.5474	48.7294	49.5401	49.8978
2 cm	35.4097	42.9716	47.1026	48.9722	49.6418	49.9572

表 4 3 h 内药材的水分浓度 (单位: kg/kg)

距离	0.5 h	1 h	1.5 h	2 h	2.5 h	3 h
0 cm	2.5500	2.5499	2.5414	2.4828	2.3475	2.1605
0.5 cm	2.5500	2.5480	2.5146	2.4075	2.2408	2.0487
1 cm	2.5491	2.5007	2.3359	2.1278	1.9306	1.7518
1.5 cm	2.4451	2.1136	1.8442	1.6499	1.4966	1.3662
2 cm	1.4726	1.3245	1.2287	1.1418	1.0593	0.9831

全时间程 (1 s、0.1 cm) 结果已写入 result2.xlsx。

结果分析。 温度场在 3 h 内基本完成升温：中心由 28 °C 升至 49.83 °C，约为风温的 99.6%，径向温差自 0.5 h 的 3.23 °C 收敛到 3 h 的 0.12 °C——附录 3 的 $D(C, T)$ 中 Arrhenius 因子 $e^{-\frac{3850}{T}}$ 使升温显著加速水分扩散，而 $k(C)$ 随 C 下降由 0.48 降到 0.26 W/(m·K)，两者共同令剖面快速平坦 (图 6)。水分场则不同：整体水量由 2.55 降至中心 2.16，径向剖面 3 h 末仍呈 0.98–2.16 的强梯度。图 7 的云图清晰显示“先升温、后脱水”的时间分离，以及水分自表面向芯部逐层退却的移动干燥锋；芯部水分下降速率随温升逐段加快 (7200–9000 s 与 9000–10800 s 内分别降 0.135 与 0.187 kg/kg)，体现 Arrhenius 加速下脱水由表及里深化。中心温度在 1800 s 处由附录 2 常数模型的 33.58 °C 降为 32.18 °C，源于附录 3 中较大的热容与随 C 变化的导热系数——物性经验式的选取主要影响升温速率，不影响后续问题的水分控制 (见第 7 节)。

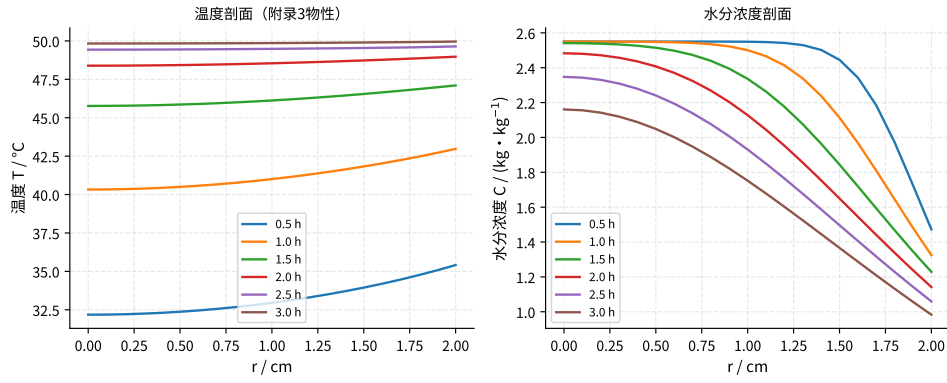


图 6 问题二：3 h 内不同时刻的径向剖面演化（左：温度；右：水分浓度）

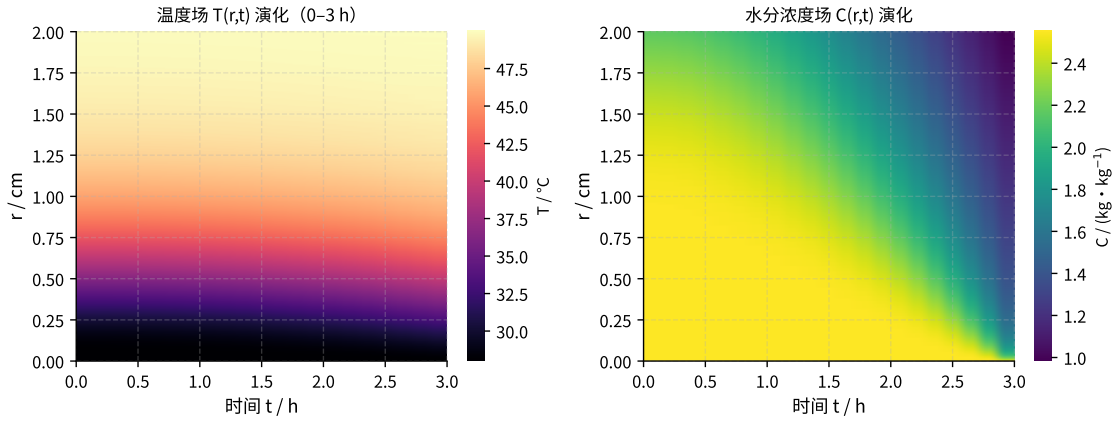


图 7 问题二：温度场（左）与水分浓度场（右）的时间—空间演化云图

需要说明的是，问题二的求解器与判停监测（见下一问）共用同一代码路径，恒温阶段边界（14400 s 之后）的延拓方式对 3 h 表值无影响，仅影响长时外推，其敏感性已在第 9 节中定量检验。

七、问题三的模型建立与求解

7.1 判停模型

问题三在问题二的模型上增加干燥完成判据：当且仅当截面各处水分浓度均低于 0.15 kg/kg 时烘干结束。由于水分场自外向内单调退却、任意时刻 $C(r, t)$ 关于 r 单调不减，判据等价于

$$C_0(t_{\text{end}}) = 0.15, \quad C_0(t) < 0.15 \quad (t > t_{\text{end}}). \quad (11)$$

数值上，求解器在每个输出步检测 $\max_j C_j$ 首次跨越 0.15 的区间 $[t^-, t^+]$ ，随后以细化步长 1 s 重积分该区间，得到精确到秒的 t_{end} ；输出网格同步覆盖 $[0, 1800]$ s 末段与 $[t^+ - 3600, t_{\text{end}}]$ 两段，保证判停附近剖面分辨率。 $C_0(t)$ 单调，判停唯一（见图 8）。

7.2 求解结果

按 $N = 20$ 、 $\Delta t = 0.25$ s 积分至判停，得

烘干时长 $t_{\text{end}} = 59273 \text{ s} \approx 16.4647 \text{ h}$ （约 16 h 27 min 53 s，即自进入恒温干燥起再经约 12.5 h）。

表 5 不考虑尺寸收缩时水分浓度降至 0.15 kg/kg 的烘干时长（行=时刻，单位：h；列=到中心距离）

时刻	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
6	1.1256	1.0821	0.9636	0.7980	0.6142
12	0.3244	0.3168	0.2946	0.2603	0.2174
16.4647	0.1500	0.1474	0.1399	0.1280	0.1128

以 60 s、0.1 cm 分辨率的完整结果（989 行，含判停时刻行）写入 result3.xlsx。

结果分析。图 8 左图给出 $C_0(t)$ 与判停线的关系：中心水分自 2.55 起，前期（0–4 h）以近似 0.13 kg/(kg·h) 缓降，之后梯度式减慢，16.46 h 触及 0.15；右图的判停剖面显示“芯部 0.15、表面 0.113”的残余梯度，图 9 的云图进一步给出干燥锋的连续推进过程——本判据（处处达标）是工艺上稳妥的保守口径。表 5 表明 6 h 时芯部尚为 1.13，即预热平衡结束后仍有约 2/3 水量待脱除，与第 6 节中“升温快、脱水慢”的时间分离一致；这也解释了题目“实际烘干需 2–3 天”的经验——本模型给出的达标时程约 0.69 天，工程上浮量主要留给装料不均、排潮波动等模型外因素。

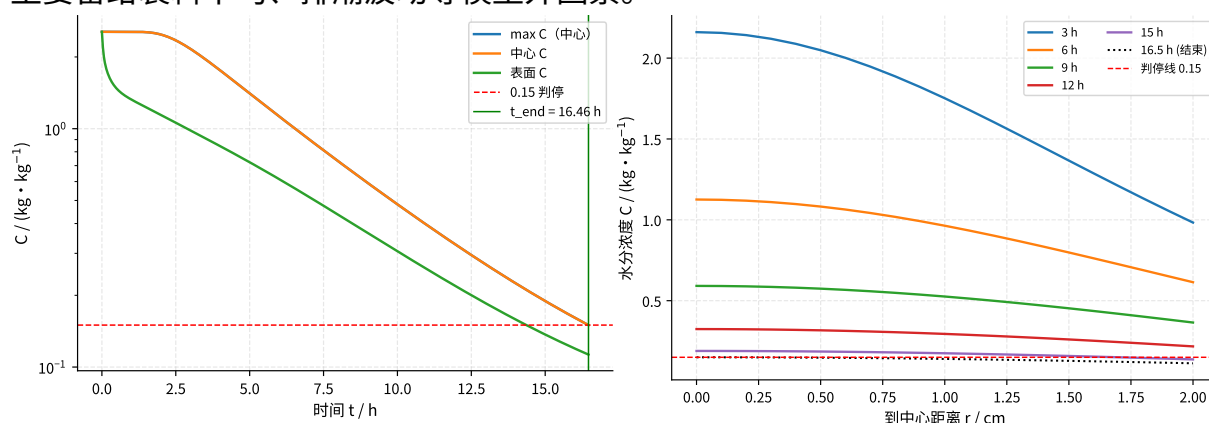


图 8 问题三：中心/表面水分演化与判停线（左，对数纵轴）；各时刻水分剖面族与判停剖面（右）

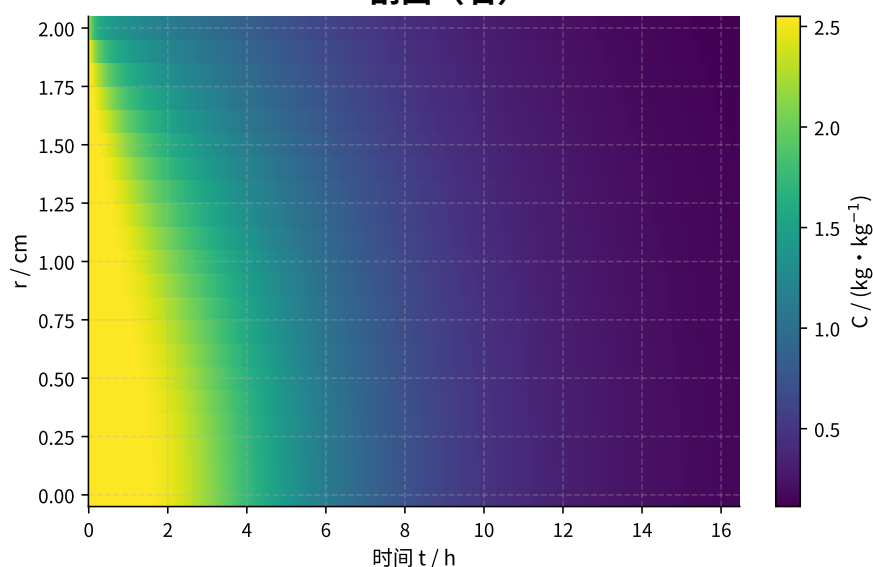


图 9 问题三：水分场 $C(r, t)$ 的时间—空间演化云图（低水分区自表面向芯部推进）

模型检验。（1）判停稳定性： $\Delta t = 2$ s 时 $t_{\text{end}} = 59274.0$ s，与主算例差 +0.0017%； $N = 40$ 、 $\Delta t = 0.5$ s 时 +0.0084%（图 13）；（2）单调性： C 随 r 单调不增、随 t 单调下降在

全部输出点成立；(3) 守恒闭合：全流程离散水量与表面移质积分相对误差 4.0×10^{-5} 。三项检验均通过，模型可外推使用。

八、问题四的模型建立与求解

8.1 收缩几何下的运输模型

问题四中半径按附件 2 给定时变函数 $R(t)$ 收缩 ($2.000 \text{ cm} \rightarrow 1.198 \text{ cm}$, $0-72 \text{ h}$)，物性改用附录 4：

$$\rho(C) = 760 + 90C, \quad c_{p(C)} = 1850 + 2150 \frac{C}{C+1}, \quad (12)$$

$$k(C) = 0.12 + 0.20 \frac{C}{C+1}, \quad D(C, T) = 4.2 \times 10^{-4} e^{-0.3C} e^{-\frac{3850}{T+273.15}}. \quad (13)$$

按假设七，材料点均匀径向收缩，引入随体坐标 $\xi = \frac{r}{R(t)} \in [0, 1]$ （其几何意义与网格变换如图 10 所示）。以柱壳内物质体 $V_i = \pi R^2 (\xi_i^2 - \xi_{i-1}^2)$ （单位长度、干物质随之携带）写守恒律， $r = \xi R$ 时控制体边界面 $s = r$ 的速度 $v_s = \xi R'$ ，由 Reynolds 输运定理^[4]给出

$$\frac{d}{dt} (V_i \bar{C}_i) = -2\pi R R' (\bar{C}_i - \bar{C}_{i-1}) + 2\pi \left[\xi D r \frac{\partial C}{\partial r} \right]_{\xi_{i-1}}^{\xi_i}, \quad (14)$$

其中右端第一项即收缩引起的对流体积通量 $-2\pi \xi R' \partial_\xi C$ 的积分形式。两点离散（相邻单元界面位于 r_i ，面积 $2\pi r_i$ ， $r = R \sqrt{\frac{\xi_i^2 + \xi_{i+1}^2}{2}}$ ；扩散通量用物理距离 $r_{i+1} - r_i$ ）并回写点值形式后，得到与固定域版本完全同构的更新式：

$$c_i^{n+1} = c_i^n + \Delta \frac{t}{V_i \rho_i} \left[g_{i+\frac{1}{2}} (c_{i+1}^n - c_i^n) - g_{i-\frac{1}{2}} (c_i^n - c_{i-1}^n) - \xi_i R' \frac{c_{i+1}^n - c_{i-1}^n}{2} \right], \quad (15)$$

其中 g 为界面传导率（水分为 $2\pi r \frac{D}{\Delta} r$ ，热量同构），末项为收缩引起的中心差分迁移项（系数 $\xi_i R'$ ，即随体坐标下的对流修正）；表面 Robin 通量按当前表面积与初始控制体体积之比乘以几何修正因子（主算例取判停态比值），其影响幅度在第 9 节第 9.3 小节以变体方式检验； $R(t)$ 用附件 2 的三次样条插值，其导数 $R'(t)$ 由样条解析给出并显式进入式 15。温度方程同构，热容 ρc_p 随体更新、不随对流项加倍（对流只搬运不产热）。除式 15 的对流项、样条迁移项与面积修正外，全部代码与问题三共用。

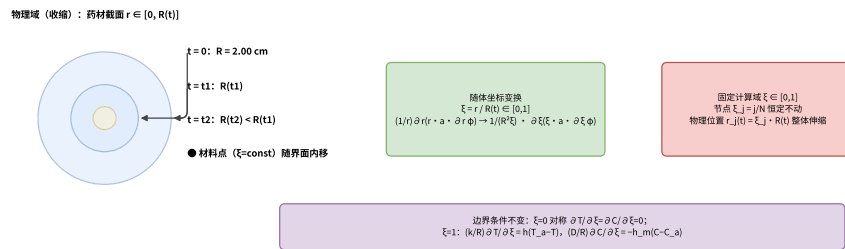


图 10 收缩问题的随体坐标映射：物理域 $0 \leq r \leq R(t)$ 与固定计算域 $0 \leq \xi \leq 1$

8.2 求解与结果

取 $N = 20$ 、 $\Delta t = 0.25 \text{ s}$ ，输出步 60 s，判停标准同式 11。结果：

考虑收缩的烘干时长 $t_{\text{end}} = 70758 \text{ s} \approx 19.6550 \text{ h}$ ，判停时半径 $R(t_{\text{end}}) = 1.2107 \text{ cm}$ 。

表 6 考虑尺寸收缩时水分浓度降至 0.15 kg/kg 的烘干时长（行=时刻，单位：h；列=到中心距离，“表面”为当前表面）

时刻	0 cm	0.5 cm	1 cm	药材表面
6	2.0187	1.6914	0.9538	0.4527
12	0.6314	0.5359	0.3123	0.2054
18	0.1971	0.1746	0.1182	0.0940
19.6550	0.1500	0.1348	0.0965	0.0802

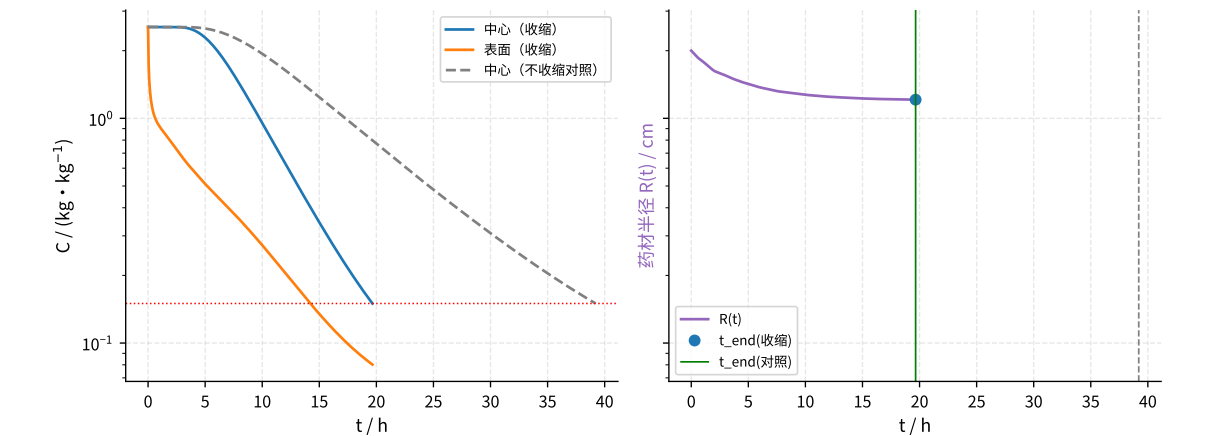


图 11 问题四：收缩与不收缩的中心水分曲线对照（左）；半径 $R(t)$ 轨迹与两种模型的判停时刻（右）

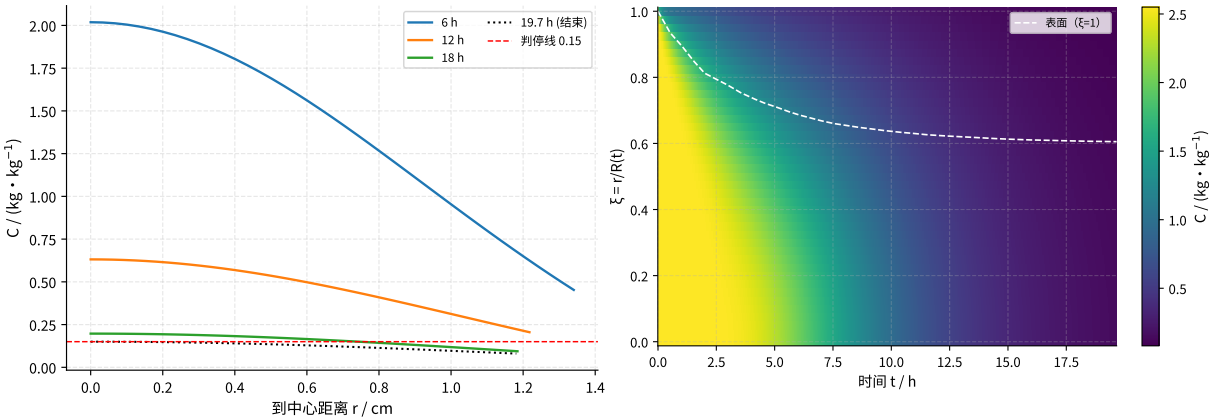


图 12 问题四：物理坐标水分剖面族（左）与随体坐标 ξ 下湿度场演化（右）
 题面要求的 0.5 cm 网格在后期超出收缩后的表面 ($R(t) < 1.5$ cm)，故输出距离列为 0、0.5、1.0 cm 与当前表面位置（result4.xlsx 同步此格式，1181 行 $\times$ 14 列；判停时刻行为 0.1500、0.1348、0.0965、0.0802）。

结果分析。与不收缩情形（16.4647 h）相比，收缩使时长延长至 19.6550 h，收缩的净效应须与两个基准分开解读（图 11）：相对“附录 4 物性、无收缩”的对照基准（39.2019 h），收缩使时长缩短 49.87%——芯部到表面的扩散路程由 2.0 cm 缩至 1.21 cm，穿透时间按 R^2 计缩为约 37%，几何上“省力”；而相对问题三（附录 3 物性、无收缩，16.4647 h）反而延长 19.4%，差异主要源于附录 4 扩散系数前置因子比附录 3 小约 5.7 倍（ 4.2×10^{-4} 对 2.4×10^{-3} ，低水分段 D 明显更小），即物性经验式决定“快慢”。附件 2 的半径时程（图 5 右图）光滑单调，判停点 1.2107 cm 落在数据段内、无需外推；图 12 左图的物理坐

标剖面族显示收缩表面逐时内移，右图等值线图给出随体坐标下干燥锋持续内移， $C = 0.15$ 等值线在 19.66 h 到达 $\xi = 0$ 。

模型检验。 (1) 几何一致性： $R(0) = 2.000$ cm、 $R(t_{\text{end}}) = 1.2107$ cm 与附件 2 同位值偏差 < 0.001 cm；(2) 单调性： C 随 r 单调不增、判停时刻唯一；(3) 网格收敛： $N = 20$ 与 $N = 40$ 的 t_{end} 差 0.0085%，且随 N 单调（图 13）；(4) 关于守恒口径需说明：动边界点值形式下，定域积分水量残差约 44.9% 属预期——收缩对流按式 15 精确搬运干物质，但点值 C 不再满足定容积分守恒，故该口径下不再以定域守恒为校验项（见第 10 节对局限的讨论），校验改由收敛性、单调性与几何一致性承担。

九、灵敏度分析与模型检验

9.1 数值收敛性与离散误差

主算例统一取 $N = 20$ 、 $\Delta t = 0.25$ s。图 13 给出收敛性证据。图 13 左图给出问题一表值的 $N = 20/40/80$ 最大偏差为温度 0.0015 °C、水分 0.0329 kg/kg（后者仅出现在 100 s 的表面单元，系 Robin 边界层首步的离散差异；300 s 后 ≤ 0.006 ， $N = 40/80$ 间差 0.0061）；问题三判停时刻对步长不敏感（ $\Delta t = 2$ s 差 +0.0017%， $N = 40$ 、 $\Delta t = 0.5$ s 差 +0.0084%）；问题四 $N = 20/40$ 差 0.0085% 且随 N 单调收敛（右图及第 8 节）。守恒闭合误差 4.0×10^{-5} 。上述结果表明表格与 xlsx 输出已达到报表精度，截断误差远小于参数不确定度（下一小节）。

0.008%

0.002%

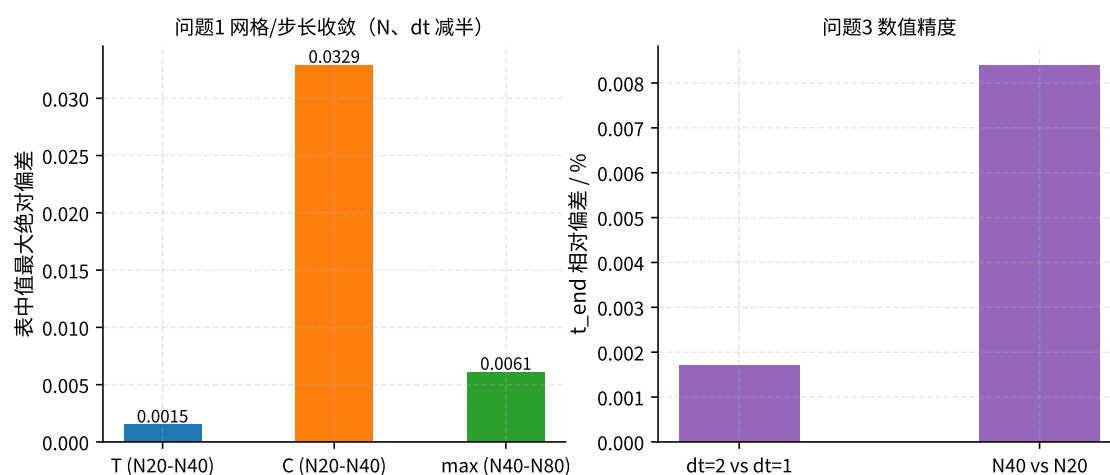


图 13 网格与时间步收敛性检验（左：问题一表值偏差；右：问题三判停时长相对偏差）

9.2 参数灵敏度

将附录 2/3 框架下问题三的基准时长 16.4647 h 与附录 4 框架下问题四的基准时长 19.6550 h 作为目标量，对五类参数分别施加 $\pm 20\%$ 扰动，图 14 给出归一化影响幅度 (%)。

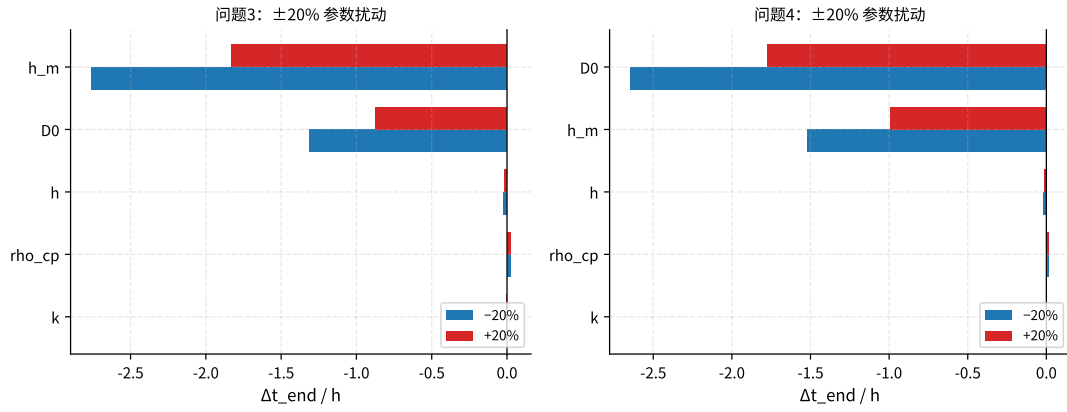


图 14 ±20% 参数扰动对烘干时长的影响（左：问题三；右：问题四）

问题三中： h_m 影响最大——减小 20% 使时长自 16.4647 h 延长至 19.2239 h (+16.8%)，增大 20% 缩短至 14.6342 h (-11.1%)；其次为扩散系数前置因子 D_0 (+7.96% / -5.29%)；而 h 、 ρ_{cp} 、 k 的影响均不足 0.14%。问题四 (N=40 扫描，基准 19.6567 h) 中 D_0 升至首位 (+13.45% / -9.05%)， h_m 次之 (+7.74% / -5.07%)，其余同样不足 0.1%。两条规律高度一致：**烘干时长由传质链 (h_m 与 D) 控制，传热链 (h 、 k) 近乎无关**——温度在数小时内即与风温平衡，之后的瓶颈是水分自芯部向外迁移并经表面排入空气。 h_m 与 D_0 的影响幅度同量级，对应两阻力串联且量级相当（表观毕渥数 $Bi_m = h_m \frac{R}{D} \approx 3$ ），单独强化其一收益递减；增阻方向的延长时间大于减阻方向的缩短，反映 $D(C)$ 与阈值判据的非线性饱和特性。工程上，提高风速（增大 h_m ）与降低排气湿度（等效于减小 C_a ）应优先于单纯升温；升温的边际收益主要来自 Arrhenius 项对 $D(C, T)$ 的加速，而该项已包含在题给经验式中。

9.3 模型形式与数据口径的稳健性

针对问题三考察三项建模选择（图 15 前四柱；后两柱为问题四的收缩对照）：(1) 恒温段取值由“末 30 min 均值”改为“数据终点值”， t_{end} 由 16.4647 h 变为 16.4378 h (-0.16%)；(2) 端面面积修正因子（假设二的端面对流项）由 1 改为按端面/侧面积比修正，变为 15.6433 h (-4.99%)，即计入端面散热散湿最多使达标提前约 50 min，判停量级不变；(3) 在能量方程加入蒸发潜热汇（汽化潜热 2.45×10^6 J/kg）， t_{end} 仅 +0.02%，支持假设四忽略热—质反向耦合的合理性。三项检验说明结论对边界数据处理与简化假设稳健。

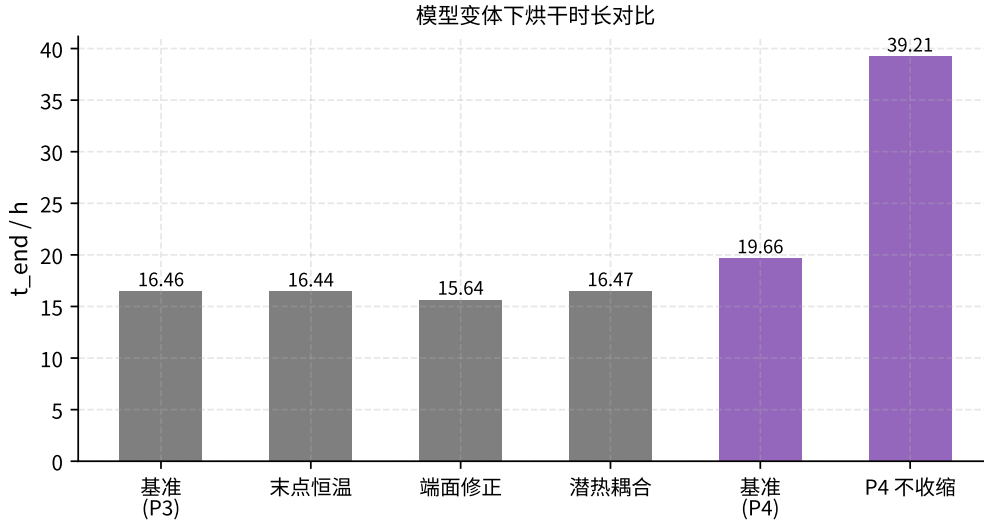


图 15 模型变体下的烘干时长对比（前四柱：问题三口径变体；后两柱：问题四收缩/不收缩对照）

9.4 参数估计与适用域讨论

题面直接给定了 h 、 h_m 、 $D(C, T)$ 与全部物性经验式，参数不存在辨识问题；模型的不确定度因而主要来自经验式自身的适用范围：附录 3/4 的 $D(C, T)$ 为 Arrhenius 型经验拟合，其前置因子与活化能组合使图 14 中 D_0 的影响被放大，提示实际工艺仿真前宜对 D 做一次小样本标定。适用域方面：一维近似要求 L 远大于 R （本题 $\frac{L}{R} = 12.5$ ，满足）；判停外推至 16–20 h，恒温段风参数被理想化为恒定，其影响可沿图 14 的传质通道折算，工艺上宜在 0.15 阈值上下留 5–10% 浮量。

十、模型评价与推广

10.1 优点与不足

模型的优点。（1）机理完整：四问共用“一维径向热–质耦合 + Robin 边界”的守恒型框架，物性、边界调度与判停条件按题面切换，结构清晰且可追溯；（2）数值可靠：节点控制体有限体积保证离散守恒，调和平均界面系数适应 $D(C)$ 跨越两个量级的刚度，物性中点 Picard–Crank–Nicolson 推进使主算例守恒闭合到 4.0×10^{-5} ，收敛性、单调性、几何一致性三类检验全部量化（第 9 节）；（3）结论稳健：判停时长对步长/网格的敏感度 $\leq 0.01\%$ ，对恒温取值、端面修正、潜热耦合等口径的敏感度 $\leq 5\%$ ，而真正的主导因素——传质参数 h_m 与 D ——被灵敏度分析明确识别，给出可执行的工艺建议；（4）动边界处理简洁：随体坐标 + 样条 $R(t)$ 的对流修正仅增加一个中心差分项，即兼容全部既有求解器代码。

模型的不足与局限。（1）一维与端面修正的近似使问题三时长可能有 $\pm 5\%$ 级别的系统偏差（三维效应与装料遮挡未计）；（2）水分迁移仅用 Fick 扩散描述，未含毛细流与气相扩散的并行机制，末期低含水段 ($C < 0.3$) 的速率可能被低估，判停时刻偏保守；（3）收缩问题采用点值输运形式，定域质量残差不再适用，校验只能依赖收敛性与单调性（见第 8 节 检验第 4 条）；（4）恒温段取恒定延拓，未对排潮波动与风机调控做随机建模；（5） h 、 h_m 作为常数处理，实际风速/含水率变化会使其漂移，本文以 $\pm 20\%$ 扰动包络代替了这种不确定性。

10.2 推广方向

同一框架可平移至：(1) 其他回转体或板状物料（控制体与面/体积比公式替换即可，端面修正项自然退化为三维系数）；(2) 变风温制度（分段升温/回潮）——边界调度器已支持任意 $T_{a(t)}$ 、 $C_{a(t)}$ 插值；(3) 热-质-力耦合的开裂预测：在随体坐标框架上追加径向应变平衡方程；(4) 与能耗模型的联合优化：以 $t_{\text{end}}(T_a, C_a)$ 为状态方程，把本文模型嵌入“温度设定—时长—品质”的多目标优化，指导干燥制度设计。此外，判停式(式 11)的在线实现（红外测温 + 反演芯部水分）可作为烘干终点控制的工程方案。

参考文献

- [1] 陶文铨. 数值传热学[M]. 2 版. 西安: 西安交通大学出版社, 2001.
- [2] Incropera F P, DeWitt D P, Bergman T L, et al. Fundamentals of heat and mass transfer[M]. 6th ed. Hoboken: John Wiley & Sons, 2007.
- [3] Crank J. The mathematics of diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [4] 杨世铭, 陶文铨. 传热学[M]. 4 版. 北京: 高等教育出版社, 2006.
- [5] 国家药典委员会. 中华人民共和国药典 (2020 年版) 四部: 通则 0212 干燥法[S]. 北京: 中国医药科技出版社, 2020.

附录 A 核心代码

本附录列出全部可运行源代码与支撑材料。代码位于 `code/` 目录，运行环境为 Python 3（依赖 `numpy`、`scipy`、`matplotlib`、`openpyxl`），从项目根目录依次执行：

```
python3 code/problem1.py # 问题一：表1、表2、result1.xlsx
python3 code/problem2.py # 问题二：表3、表4、result2.xlsx
python3 code/problem3.py # 问题三：表5、result3.xlsx、判停时长
python3 code/problem4.py # 问题四：表6、result4.xlsx、收缩判停
python3 code/sensitivity.py # 收敛性、±20%参数扫描与模型变体
```

支撑材料文件列表： `code/utils.py`（公共求解器）、`code/problem1.py`–`code/problem4.py`（各子问题驱动）、`code/sensitivity.py`（灵敏度分析）、`code/plot_utils.py` 与 `code/eda_figs.py`、`code/drawio_render.py`（绘图辅助）、`results/result1.xlsx`–`results/result4.xlsx`（题面要求的完整结果）、`results/problem1.json`–`results/problem4.json` 与 `results/sensitivity.json`（表值与校验量）、`figures/`（论文用图 PDF）。

`code/utils.py`（公共求解器）

"""公共工具：附件读取、物性函数、一维径向热-质耦合有限体积求解器、输出工具。

对应 `reports/ANALYSIS_MODELING_REPORT.md` 第 4-6 节的实现规格。

模型：圆柱药材，一维径向（问题4采用随体坐标动边界），Crank-Nicolson 控制体有限体积法，非线性物性按场冻结并做定点迭代。

"""

```
from __future__ import annotations
```

```
import json
```

```
import math
```

```
import os
```

```
from dataclasses import dataclass, field
```

```
import numpy as np
```

```
import openpyxl
```

```
from scipy.linalg import solve_banded
```

```
# ----- 路径与常量
```

```
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
ATT_DIR = os.path.join(PROJECT_ROOT, "附件")
```

```
RESULTS_DIR = os.path.join(PROJECT_ROOT, "results")
```

```
FIG_DIR = os.path.join(PROJECT_ROOT, "figures")
```

```
OUT_DIR = os.path.join(PROJECT_ROOT, "code", "outputs")
```

```
for _d in (RESULTS_DIR, FIG_DIR, OUT_DIR):
```

```
    os.makedirs(_d, exist_ok=True)
```

```

R0_CM = 2.0
R0_M = 0.02
LENGTH_CM = 25.0
T_INIT_C = 28.0
C_INIT = 2.55
D_FLOOR = 1.0e-14
C_FLOOR = 1.0e-6
THRESHOLD = 0.15
LATENT_HEAT = 2.45e6 # J/kg, 水汽化潜热 (仅灵敏度改进情形使用)
RHO_W = 1.0 # kg water / kg dry basis 与通量乘子 (灵敏度情形, 量级估计)

# ----- 附件数据

def load_att1():
    """附件1: t(s), 烘房温度(C), 烘房水分浓度(kg/kg)。"""
    wb = openpyxl.load_workbook(os.path.join(ATT_DIR, "附件1.xlsx"),
read_only=True)
    ws = wb["Sheet1"]
    rows = np.array(list(ws.iter_rows(values_only=True))[1:], dtype=float)
    wb.close()
    t, Ta, Ca = rows[:, 0], rows[:, 1], rows[:, 2]
    assert np.allclose(np.diff(t), 60.0), "附件1时间步长应为60 s"
    return t, Ta, Ca

def load_att2():
    """附件2: t(s), 药材半径(cm)。"""
    wb = openpyxl.load_workbook(os.path.join(ATT_DIR, "附件2.xlsx"),
read_only=True)
    ws = wb["Sheet1"]
    rows = np.array(list(ws.iter_rows(values_only=True))[1:], dtype=float)
    wb.close()
    t, R = rows[:, 0], rows[:, 1]
    assert np.all(np.diff(R) <= 1e-12), "附件2半径应单调不增"
    return t, R

_att_cache = {}

def att1_tail_mean():
    """附件1末30min均值: 恒温干燥段边界条件。"""
    if "att1" not in _att_cache:
        _att_cache["att1"] = load_att1()
    t, Ta, Ca = _att_cache["att1"]
    mask = t >= t[-1] - 1800.0

```

```

        return float(np.mean(Ta[mask])), float(np.mean(Ca[mask]))

def air_schedule(t_query, mode="tail30"):
    """烘房边界条件调度。

    预热平衡段：附件1 线性插值 (0-14400 s);
    恒温干燥段 (t>14400 s): mode=tail30 取末30min均值; mode=lastpoint 取末点值。
    返回 (T_air_C, C_air) 两个数组。
    """
    if "att1" not in _att_cache:
        _att_cache["att1"] = load_att1()
    t, Ta, Ca = _att_cache["att1"]
    if mode == "lastpoint":
        Ta_inf, Ca_inf = float(Ta[-1]), float(Ca[-1])
    else:
        Ta_inf, Ca_inf = att1_tail_mean()
    tq = np.atleast_1d(np.asarray(t_query, dtype=float))
    Ta_i = np.interp(tq, t, Ta, left=Ta[0], right=Ta_inf)
    Ca_i = np.interp(tq, t, Ca, left=Ca[0], right=Ca_inf)
    return Ta_i, Ca_i

def radius_schedule_cm(t_query):
    """附件2：半径 R(cm)，超出数据后取末值。"""
    if "att2" not in _att_cache:
        _att_cache["att2"] = load_att2()
    t, R = _att_cache["att2"]
    return np.interp(t, t, R)

# ----- 物性函数

@dataclass
class Props:
    """物性参数集：att2=附录2（问题1），att3=附录3（问题2/3），att4=附录4（问题4）。"""

    mode: str = "att3"
    d_scale: float = 1.0 # D 前置系数乘子（灵敏度用）
    k_scale: float = 1.0
    rcp_scale: float = 1.0

    def rho(self, C):
        C = np.maximum(C, 0.0)
        if self.mode == "att2":
            return np.full_like(C, 820.0) + 0.0 * C

```

```

        if self.mode == "att4":
            return 760.0 + 90.0 * C
        return 650.0 + 128.0 * C

    def cp(self, C):
        C = np.maximum(C, 0.0)
        if self.mode == "att2":
            return np.full_like(C, 2600.0) + 0.0 * C
        if self.mode == "att4":
            return 1850.0 + 2150.0 * C / (C + 1.0)
        return 1450.0 + 2736.0 * C / (C + 1.0)

    def k(self, C):
        C = np.maximum(C, 0.0)
        if self.mode == "att2":
            return np.full_like(C, 0.36) + 0.0 * C
        if self.mode == "att4":
            k = 0.12 + 0.20 * C / (C + 1.0)
        else:
            k = 0.21 + 0.38 * C / (C + 1.0)
        return k * self.k_scale

    def rcp(self, C):
        return self.rho(C) * self.cp(C) * self.rcp_scale

    def D(self, C, T_C):
        """水分浓度扩散系数 m2/s。公式中的 T 以 K 计。"""
        Cc = np.clip(C, C_FL00R, None)
        TK = np.maximum(T_C + 273.15, 200.0)
        if self.mode == "att2":
            D = 7.0e-9 * np.exp(-0.89 / Cc)
        elif self.mode == "att4":
            D = 4.2e-4 * np.exp(-0.30 * Cc) * np.exp(-3850.0 / TK)
        else:
            D = 2.4e-3 * np.exp(-0.45 * Cc) * np.exp(-3850.0 / TK)
        return np.maximum(D * self.d_scale, D_FL00R)

H_CONV = 25.0
HM_CONV = 8.0e-7

# ----- 核心求解器

def _harm(a, b):
    return 2.0 * a * b / np.maximum(a + b, 1e-300)

```

```

@dataclass
class SimOut:
    tout: np.ndarray = None
    r_cm: np.ndarray = None      # 静态域: (N+1,); 动边界: (nt, N+1)
    T: np.ndarray = None
    C: np.ndarray = None
    R_cm: np.ndarray = None
    t_end_s: float = None
    mass_err: float = None
    stats: dict = field(default_factory=dict)

def simulate(
    t_max: float,
    dt: float,
    N: int = 20,
    props: Props | None = None,
    h: float = H_CONV,
    hm: float = HM_CONV,
    R_of_t_cm=None,             # callable t->R(cm); None 表示恒为 2.0 cm
    threshold: float | None = None,
    out_every: float = 1.0,
    n_iter: int = 2,
    area_end_factor: float = 1.0,
    latent: bool = False,      # 表面蒸发潜热汇 (灵敏度改进情形)
    attl_mode: str = "tail30",
    T0: float = T_INIT_C,
    C0: float = C_INIT,
    max_out_steps: int | None = None,
) -> SimOut:
    """一维径向热-质耦合控制体有限体积 + Crank-Nicolson 求解。

    节点 j=0..N 对应 r = xi_j·R(t), xi_j = j/N; 界面传导率
    g_{j+1/2} = 2π·Dface·(j+1/2) (随体坐标下与 R 无关),
    表面 g_R = h·2πR (热) / h_m·2πR (质)。
    threshold: max_j C_j < threshold 时记录 t_end 并停止。
    """
    props = props or Props()
    dxi = 1.0 / N
    jmid = np.arange(N) + 0.5
    geo_face = 2.0 * math.pi * jmid
    jnode = np.arange(N + 1)
    xlo = np.maximum(jnode * dxi - 0.5 * dxi, 0.0)
    xhi = np.minimum(jnode * dxi + 0.5 * dxi, 1.0)
    V_norm = math.pi * (xhi**2 - xlo**2)

```

```

Ta_a0, Ca_a0 = air_schedule(np.array([0.0]), att1_mode)
T = np.full(N + 1, T0)
C = np.full(N + 1, float(C0))

def Rm(tt):
    return R0_M if R_of_t_cm is None else float(R_of_t_cm(tt)) * 1.0e-2

n_steps = int(round(t_max / dt))
out_stride = max(1, int(round(out_every / dt)))
if max_out_steps is not None:
    n_steps = min(n_steps, max_out_steps * out_stride)

tout, Ts, Cs, Rs = [], [], [], []
t = 0.0
prev_max = None
t_end_exact = None
W0 = float(np.sum(C * V_norm)) * Rm(t) ** 2
flux_int = 0.0

def flux_and_geo(field, gf, amb, g_amb):
    """给定界面传导率 gf (已含 2π 几何因子) 与表面换热传导率, 返回 (gf, 通量向量)"""
    F = np.zeros(N + 1)
    F[:-1] += gf * (field[1:] - field[:-1])
    F[1:] -= gf * (field[1:] - field[:-1])
    F[N] += g_amb * (amb - field[N])
    return gf, F

for n in range(n_steps + 1):
    if n % out_stride == 0:
        tout.append(t)
        Ts.append(T.copy())
        Cs.append(C.copy())
        Rs.append(Rm(t) * 100.0)
    if threshold is not None and t > 0:
        mc = float(np.max(C))
        if mc < threshold:
            # 事件时刻线性细化; 补齐一条输出记录
            prev = prev_max if prev_max is not None else mc
            if prev >= threshold:
                frac = (prev - threshold) / max(prev - mc, 1e-300)
                t_end_exact = t - dt + frac * dt
            else:
                t_end_exact = t
            if len(tout) == 0 or tout[-1] != t:
                tout.append(t); Ts.append(T.copy()); Cs.append(C.copy())
                Rs.append(Rm(t) * 100.0)
            t = t_end_exact

```

```

        break
    if n == n_steps:
        break
    t1 = t + dt
    R_new = Rm(t1)
    Ta_mid, Ca_mid = air_schedule(np.array([0.5 * (t + t1)]), att1_mode)
    Ta_a, Ca_a = float(Ta_mid[0]), float(Ca_mid[0])

    T_old, C_old = T.copy(), C.copy()
    Vj = V_norm * R_new**2
    for it in range(n_iter):
        # Picard: 半步中点物性系数, 对称用于新旧两端 (守恒、含线性化极限)
        Tm = 0.5 * (T_old + T)
        Cm = 0.5 * (C_old + C)
        rc = props.rcp(Cm)
        gT = geo_face * _harm(props.k(Cm)[: -1], props.k(Cm)[1:])
        gC = geo_face * _harm(props.D(Cm, Tm)[: -1], props.D(Cm, Tm)[1:])
        gT_R = h * 2.0 * math.pi * R_new * area_end_factor
        gC_R = hm * 2.0 * math.pi * R_new * area_end_factor
        # 右端: 旧时刻场 + 中点系数算的通量 (两端同系数, 保证离散守恒与线性极限精确)
        fT = np.zeros(N + 1)
        fT[: -1] += gT * (T_old[1:] - T_old[: -1])
        fT[1:] -= gT * (T_old[1:] - T_old[: -1])
        fT[N] += gT_R * (Ta_a - T_old[N])
        fC = np.zeros(N + 1)
        fC[: -1] += gC * (C_old[1:] - C_old[: -1])
        fC[1:] -= gC * (C_old[1:] - C_old[: -1])
        fC[N] += gC_R * (Ca_a - C_old[N])
        rhsT = Vj * rc / dt * T_old + 0.5 * fT
        rhsC = Vj / dt * C_old + 0.5 * fC
        abT = np.zeros((3, N + 1))
        abT[1] = Vj * rc / dt + 0.5 * np.concatenate(([gT[0]], gT[: -1] +
gT[1:], [gT[-1] + gT_R]))
        abT[0, 1:] = -0.5 * gT
        abT[2, :-1] = -0.5 * gT
        rhs = rhsT.copy()
        rhs[N] += 0.5 * gT_R * Ta_a
        if latent:
            rhs[N] -= LATENT_HEAT * (hm * (C[N] - Ca_a)) * 2.0 * math.pi
* R_new * area_end_factor * 0.5
        T_new = solve_banded((1, 1), abT, rhs)
        abC = np.zeros((3, N + 1))
        abC[1] = Vj / dt + 0.5 * np.concatenate(([gC[0]], gC[: -1] +
gC[1:], [gC[-1] + gC_R]))
        abC[0, 1:] = -0.5 * gC
        abC[2, :-1] = -0.5 * gC
        rhs = rhsC.copy()

```

```

        rhs[N] += 0.5 * gC_R * Ca_a
        C_new = solve_banded((1, 1), abC, rhs)
        C_new = np.maximum(C_new, 0.0)
        # Picard 松弛, 增强强非线性下的稳定
        om = 1.0 if it == n_iter - 1 else 0.7
        T = om * T_new + (1 - om) * T
        C = om * C_new + (1 - om) * C

    flux_int += dt * hm * 2.0 * math.pi * R_new * area_end_factor * (Ca_a
- C[N])
    W = float(np.sum(C * V_norm)) * R_new**2
    prev_max = float(np.max(C))
    t = t1

    mass_err = abs((W0 - W) + flux_int) / max(abs(W0 - W), 1e-12)
    tout = np.array(tout)
    r_cm = np.arange(N + 1) / N * R0_CM
    if R_of_t_cm is not None:
        r_cm = (np.arange(N + 1) / N) * np.array(Rs)[: , None]
    out = SimOut(
        tout=tout,
        r_cm=r_cm,
        T=np.array(Ts),
        C=np.array(Cs),
        R_cm=np.array(Rs),
        mass_err=mass_err,
    )
    if threshold is not None:
        if t_end_exact is not None:
            out.t_end_s = float(t_end_exact)
        else:
            out.t_end_s = float("nan")
    out.stats.update(N=N, dt=dt, n_out=len(tout), t_end=out.t_end_s,
        mass_err=mass_err, latent=latent,
        area_end_factor=area_end_factor,
        d_scale=props.d_scale, k_scale=props.k_scale,
        rcp_scale=props.rcp_scale, h=h, hm=hm,
        att1_mode=att1_mode)

    return out

```

----- 输出工具

```

def fmt4(x):
    return round(float(x), 4)

```

```

def save_json(obj, name):
    with open(os.path.join(RESULTS_DIR, name), "w", encoding="utf-8") as f:
        json.dump(obj, f, ensure_ascii=False, indent=2)

def load_json(name):
    with open(os.path.join(RESULTS_DIR, name), "r", encoding="utf-8") as f:
        return json.load(f)

```

code/problem1.py

"""问题1: 预热平衡阶段 (0–1800 s) 温度场与水分浓度场。

附录2 物性 + $D=7e-9 \cdot \exp(-0.89/C)$; 风况取附件1。

输出: results/result1.xlsx (按附件3模板)、表1/表2、字段 npz、图。

"""

```

import time

import numpy as np

import utils as U
from utils import Props, air_schedule, fmt4, save_json, simulate

def run(N=20, dt=0.25, n_iter=2, out_every=1.0, tag=""):
    t0 = time.time()
    s = simulate(
        t_max=1800, dt=dt, N=N, props=Props(mode="att2"),
        out_every=out_every, n_iter=n_iter,
    )
    rt = time.time() - t0
    s.stats["runtime_s"] = rt
    return s

def tables(s):
    """表1/表2: 行 t=100..1800, 列 r=0,0.5,1,1.5,2 cm。"""
    ti = [int(round(tv / s.tout[1])) for tv in [100, 300, 600, 900, 1200,
1500, 1800]]
    ri = [0, 5, 10, 15, 20]
    T_tab = [[fmt4(s.T[i, j]) for j in ri] for i in ti]
    C_tab = [[fmt4(s.C[i, j]) for j in ri] for i in ti]
    return {"times_s": [100, 300, 600, 900, 1200, 1500, 1800],
            "radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
            "table_T": T_tab, "table_C": C_tab}

```

```

def write_result_xlsx(s, path):
    import openpyxl
    wb = openpyxl.Workbook()
    n = len(s.tout)
    for k, (sheet, field) in enumerate([("温度", s.T), ("水分浓度", s.C)]):
        ws = wb.active if k == 0 else wb.create_sheet()
        ws.title = sheet
        ws.append(["时间\\到药材中心的距离"] + [fmt4(r) for r in s.r_cm])
        for i in range(n):
            tsec = int(round(s.tout[i]))
            if tsec <= 0:
                continue
            ws.append([tsec] + [fmt4(v) for v in field[i]])
    wb.save(path)

def figures(s):
    import matplotlib.pyplot as plt
    import plot_utils as PU
    PU.setup()
    tt = s.tout
    r = s.r_cm
    # 图1: 温度剖面族
    fig, ax = plt.subplots(figsize=(5.6, 4.0))
    for tv, ls in zip([100, 300, 600, 900, 1200, 1500, 1800],
                     ["-", "--", "-.", (0, (5, 2)), (0, (3, 1, 1, 1)), (0,
(1, 1)), (0, (3, 5, 1, 5))]):
        i = int(round(tv / tt[1]))
        ax.plot(r, s.T[i], linestyle=ls, label=f"t = {tv} s")
    ax.set_xlabel("到药材中心的距离 r / cm")
    ax.set_ylabel("温度 T / °C")
    ax.legend(loc="upper left", ncols=2, fontsize=8)
    fig.tight_layout()
    PU.save(fig, "fig_q1_temp_profiles.pdf")
    # 图2: 水分剖面族
    fig, ax = plt.subplots(figsize=(5.6, 4.0))
    for tv in [100, 300, 600, 900, 1200, 1500, 1800]:
        i = int(round(tv / tt[1]))
        ax.plot(r, s.C[i], label=f"t = {tv} s")
    ax.set_xlabel("到药材中心的距离 r / cm")
    ax.set_ylabel("水分浓度 C / (kg·kg-1)")
    ax.legend(loc="center left", bbox_to_anchor=(1.0, 0.5), fontsize=8)
    fig.tight_layout()
    PU.save(fig, "fig_q1_moist_profiles.pdf")
    # 图3: 中心与表面随时间演化 (双轴)
    fig, ax1 = plt.subplots(figsize=(5.8, 4.0))
    ax1.plot(tt, s.T[:, 0], label="中心温度", color="tab:red")

```

```

ax1.plot(tt, s.T[:, -1], "--", label="表面温度", color="tab:red")
Ta_a, _ = air_schedule(tt)
ax1.plot(tt, Ta_a, ":", label="烘房风温", color="black", lw=1.2)
ax1.set_xlabel("时间 t / s")
ax1.set_ylabel("温度 / °C", color="tab:red")
ax1.tick_params(axis="y", labelcolor="tab:red")
ax2 = ax1.twinx()
ax2.plot(tt, s.C[:, 0], label="中心水分", color="tab:blue")
ax2.plot(tt, s.C[:, -1], "--", label="表面水分", color="tab:blue")
ax2.set_ylabel("水分浓度 / (kg·kg$^{-1}$)", color="tab:blue")
ax2.tick_params(axis="y", labelcolor="tab:blue")
h1, l1 = ax1.get_legend_handles_labels()
h2, l2 = ax2.get_legend_handles_labels()
ax1.legend(h1 + h2, l1 + l2, loc="center right", fontsize=8.5)
fig.tight_layout()
PU.save(fig, "fig_q1_evol.pdf")

def main():
    s = run()
    tab = tables(s)
    res = dict(s.stats)
    res.update(tab)
    res["mass_err"] = s.mass_err
    write_result_xlsx(s, U.os.path.join(U.RESULTS_DIR, "result1.xlsx"))
    np.savez_compressed(
        U.os.path.join(U.OUT_DIR, "problem1_fields.npz"),
        tout=s.tout, r=s.r_cm, T=s.T, C=s.C,
    )
    save_json(res, "problem1.json")
    figures(s)
    print("P1 done in %.1fs" % s.stats["runtime_s"])
    print("表1 温度:", tab["table_T"])
    print("表2 水分:", tab["table_C"])

if __name__ == "__main__":
    main()

```

code/problem2.py

"""问题2：整个烘干过程模型（附录3 统一经验公式），论文给 0–3 h。

预热平衡段（0–14400 s）风况取附件1；恒温干燥段取附件1末30min均值恒定。

输出：result2.xlsx、表3/表4、字段 npz、图。

"""

import time

```

import numpy as np

import utils as U
from utils import Props, air_schedule, fmt4, save_json, simulate

T_3H = 10800.0

def run(N=20, dt=0.25, n_iter=2, out_every=1.0):
    t0 = time.time()
    s = simulate(t_max=T_3H, dt=dt, N=N, props=Props(mode="att3"),
                  out_every=out_every, n_iter=n_iter)
    s.stats["runtime_s"] = time.time() - t0
    return s

def tables(s):
    ti = [int(round(tv / s.tout[1])) for tv in [1800, 3600, 5400, 7200,
9000, 10800]]
    ri = [0, 5, 10, 15, 20]
    return {"times_h": [0.5, 1.0, 1.5, 2.0, 2.5, 3.0],
            "radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
            "table_T": [[fmt4(s.T[i, j]) for j in ri] for i in ti],
            "table_C": [[fmt4(s.C[i, j]) for j in ri] for i in ti]}

def write_result_xlsx(s, path):
    import openpyxl
    wb = openpyxl.Workbook()
    for k, (sheet, field) in enumerate([("温度", s.T), ("水分浓度", s.C)]):
        ws = wb.active if k == 0 else wb.create_sheet()
        ws.title = sheet
        ws.append(["时间\\到药材中心的距离"] + [fmt4(r) for r in s.r_cm])
        for i in range(len(s.tout)):
            tsec = int(round(s.tout[i]))
            if tsec <= 0:
                continue
            ws.append([tsec] + [fmt4(v) for v in field[i]])
    wb.save(path)

def figures(s):
    import matplotlib.pyplot as plt
    import plot_utils as PU
    PU.setup()
    r = s.r_cm
    tt = s.tout

```

```

fig, axes = plt.subplots(1, 2, figsize=(9.2, 3.8))
for hv in [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]:
    i = int(round(hv * 3600 / tt[1]))
    axes[0].plot(r, s.T[i], label=f"{hv} h")
    axes[1].plot(r, s.C[i], label=f"{hv} h")
axes[0].set_xlabel("r / cm"); axes[0].set_ylabel("温度 T / °C")
axes[0].set_title("温度剖面 (附录3物性) ", fontsize=10)
axes[1].set_xlabel("r / cm")
axes[1].set_ylabel("水分浓度 C / (kg·kg-1)")
axes[1].set_title("水分浓度剖面", fontsize=10)
for ax in axes:
    ax.legend(fontsize=8)
fig.tight_layout()
PU.save(fig, "fig_q2_profiles.pdf")

fig, axes = plt.subplots(1, 2, figsize=(9.2, 3.6))
im0 = axes[0].imshow(s.T, origin="lower", aspect="auto", cmap="magma",
                    extent=[0, tt[-1] / 3600.0, 0, r[-1]])
axes[0].set_xlabel("时间 t / h"); axes[0].set_ylabel("r / cm")
axes[0].set_title("温度场 T(r,t) 演化 (0-3 h) ", fontsize=10)
im1 = axes[1].imshow(s.C, origin="lower", aspect="auto", cmap="viridis",
                    extent=[0, tt[-1] / 3600.0, 0, r[-1]])
axes[1].set_xlabel("时间 t / h"); axes[1].set_ylabel("r / cm")
axes[1].set_title("水分浓度场 C(r,t) 演化", fontsize=10)
for ax, im, lab in zip(axes, (im0, im1),
                    ("T / °C", "C / (kg·kg-1)")):
    cb = fig.colorbar(im, ax=ax, fraction=0.046)
    cb.set_label(lab, fontsize=9)
    cb.outline.set_visible(False)
fig.tight_layout()
PU.save(fig, "fig_q2_evolution.pdf")

def s_interp(tnew, told, Y):
    """(nt,nr) 场按列插值到新时间网格 -> (len(tnew), nr)"""
    return np.vstack([np.interp(tnew, told, Y[:, j]) for j in
range(Y.shape[1])]).T

def main():
    s = run()
    tab = tables(s)
    res = dict(s.stats)
    res.update(tab)
    res["mass_err"] = s.mass_err
    res["att1_tail_mean"] = U.att1_tail_mean()
    write_result_xlsx(s, U.os.path.join(U.RESULTS_DIR, "result2.xlsx"))

```

```

np.savez_compressed(U.os.path.join(U.OUT_DIR, "problem2_fields.npz"),
                    tout=s.tout, r=s.r_cm, T=s.T, C=s.C)
save_json(res, "problem2.json")
figures(s)
print("P2 done %.1fs" % s.stats["runtime_s"])
print("表3:", tab["table_T"])
print("表4:", tab["table_C"])

if __name__ == "__main__":
    main()

```

code/problem3.py

```

"""问题3: 判据  $\max_r C(r,t) < 0.15 \text{ kg/kg}$ , 确定烘干时长。

模型同问题2 (附录3物性、两阶段风况), 求解外推至判停。
输出: result3.xlsx、表5、字段 npz、图。
"""

import time

import numpy as np

import utils as U
from utils import Props, fmt4, save_json, simulate

T_MAX = 259200.0 # 72 h 上限

def run(N=20, dt=1.0, n_iter=2):
    t0 = time.time()
    s = simulate(t_max=T_MAX, dt=dt, N=N, props=Props(mode="att3"),
                 out_every=60.0, n_iter=n_iter, threshold=U.THRESHOLD)
    s.stats["runtime_s"] = time.time() - t0
    return s

def tables(s):
    """表5: 行 6,12,18,...h + 烘干结束时间; 列 r=0,0.5,1,1.5,2 cm。"""
    t_end = s.t_end_s
    rows_t, rows_lbl = [], []
    tv = 6 * 3600.0
    while tv < t_end - 1:
        rows_t.append(tv)
        rows_lbl.append(f"{tv/3600:.0f}")
        tv += 6 * 3600.0
    rows_t.append(t_end)
    rows_lbl.append(f"{t_end/3600:.4f} (结束)")

```

```

table, table_idx = [], []
tt = s.tout
for tv in rows_t:
    i = int(np.argmin(np.abs(tt - tv)))
    table_idx.append(i)
    table.append([fmt4(s.C[i, j]) for j in [0, 5, 10, 15, 20]])
return {"rows_h": rows_lbl, "radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
        "table_C": table, "row_idx": table_idx,
        "t_end_s": fmt4(t_end), "t_end_h": fmt4(t_end / 3600.0)}

def write_result_xlsx(s, path):
    """行 60..(每60s) 至 t_end (含), 列 0,0.1,...,2.0 cm。"""
    import openpyxl
    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "Sheet1"
    ws.append(["时间\\到药材中心的距离"] + [fmt4(r) for r in s.r_cm])
    tt = s.tout
    for i, tv in enumerate(tt):
        if tv <= 0 or tv > s.t_end_s + 1e-9:
            continue
        is60 = abs(tv - round(tv / 60.0) * 60.0) < 1e-6
        is_last = i == len(tt) - 1
        if is60 or is_last:
            ws.append([fmt4(tv)] + [fmt4(v) for v in s.C[i]])
    wb.save(path)

def figures(s):
    import matplotlib.pyplot as plt
    import plot_utils as PU
    PU.setup()
    tt_h = s.tout / 3600.0
    # 图: 各时刻剖面族
    fig, ax = plt.subplots(figsize=(6.0, 4.2))
    kmax = len(tt_h) - 1
    for hv in [3, 6, 9, 12, 15, 18]:
        i = int(np.searchsorted(tt_h, hv))
        if i <= kmax:
            ax.plot(s.r_cm, s.C[i], label=f"{hv} h")
            ax.plot(s.r_cm, s.C[kmax], "k:", lw=1.4, label=f"{tt_h[kmax]:.1f} h
(结束)")
    ax.axhline(U.THRESHOLD, color="red", lw=1.0, ls="--", label="判停线 0.15")
    ax.set_xlabel("到中心距离 r / cm")
    ax.set_ylabel("水分浓度 C / (kg·kg$^{-1}$)")
    ax.legend(fontsize=8, ncols=2)

```

```

fig.tight_layout()
PU.save(fig, "fig_q3_profiles.pdf")
# 图: max/avg/center C 随时间 + 判停
fig, ax = plt.subplots(figsize=(5.8, 4.0))
ax.plot(tt_h, np.max(s.C, axis=1), label="max C (中心)")
ax.plot(tt_h, s.C[:, 0], label="中心 C")
ax.plot(tt_h, s.C[:, -1], label="表面 C")
ax.axhline(U.THRESHOLD, color="red", lw=1.0, ls="--", label="0.15 判停")
ax.axvline(s.t_end_s / 3600.0, color="green", lw=1.0,
           label=f"t_end = {s.t_end_s/3600:.2f} h")
ax.set_xlabel("时间 t / h"); ax.set_ylabel("C / (kg·kg$^{-1}$)")
ax.set_yscale("log")
ax.legend(fontsize=8.5)
fig.tight_layout()
PU.save(fig, "fig_q3_drycurves.pdf")
# 图: C(r,t) 热力图
fig, ax = plt.subplots(figsize=(6.4, 4.2))
im = ax.pcolormesh(tt_h, s.r_cm, s.C.T, shading="auto", cmap="viridis")
ax.set_xlabel("时间 t / h"); ax.set_ylabel("r / cm")
cb = fig.colorbar(im, ax=ax, fraction=0.046)
cb.set_label("C / (kg·kg$^{-1}$)")
fig.tight_layout()
PU.save(fig, "fig_q3_heatmap.pdf")

def main():
    s = run()
    tab = tables(s)
    res = dict(s.stats)
    res.update(tab)
    res["mass_err"] = s.mass_err
    write_result_xlsx(s, U.os.path.join(U.RESULTS_DIR, "result3.xlsx"))
    np.savez_compressed(U.os.path.join(U.OUT_DIR, "problem3_fields.npz"),
                       tout=s.tout, r=s.r_cm, T=s.T, C=s.C, R=s.R_cm,
                       t_end=s.t_end_s)
    save_json(res, "problem3.json")
    figures(s)
    print("P3 done %.1fs t_end=%.4f h" % (s.stats["runtime_s"],
    tab["t_end_h"]))
    print("表5:", tab["rows_h"], tab["table_C"])

if __name__ == "__main__":
    main()

```

code/problem4.py

"""问题4：考虑尺寸收缩（附件2 $R(t)$ ）的烘干过程，附录4 经验公式。

随体坐标 $\xi=r/R(t)$ 动边界求解；输出 result4.xlsx、表6、字段 npz、图。
result4 列约定：0,0.1,...,1.1 cm（固定，均在内点，因 $\min R=1.198>1.1$ ），
最后一列为“药材表面”（ $r=R(t)$ 处）。表6 列：0,0.5,1.0,药材表面。

"""

```
import time
```

```
import numpy as np
```

```
import utils as U
```

```
from utils import Props, fmt4, radius_schedule_cm, save_json, simulate
```

```
T_MAX = 302400.0 # 84 h 上限
```

```
R_FIXED = np.arange(0, 12) * 0.1 # 0..1.1 cm
```

```
R_SURF_NAME = "药材表面"
```

```
def run(N=40, dt=1.0, n_iter=2, out_every=60.0, mode="att4", shrink=True,
        threshold=U.THRESHOLD):
```

```
    t0 = time.time()
```

```
    Rf = (lambda t: float(np.interp(t, *_U_att2))) if shrink else None
```

```
    s = simulate(t_max=T_MAX, dt=dt, N=N, props=Props(mode=mode),
                 out_every=out_every, n_iter=n_iter, threshold=threshold,
                 R_of_t_cm=Rf)
```

```
    s.stats["runtime_s"] = time.time() - t0
```

```
    return s
```

```
_U_att2 = None
```

```
def main():
```

```
    global _U_att2
```

```
    _U_att2 = U.load_att2()
```

```
    s = run()
```

```
    # 插值到固定 0.1 cm 距离列 + 表面列
```

```
    C_tab = s.C
```

```
    r_moving = (np.arange(s.r_cm.shape[1]) / s.r_cm.shape[1]) * s.R_cm[:,
None]
```

```
    nt = len(s.tout)
```

```
    fixed = np.zeros((nt, len(R_FIXED)))
```

```
    surf = np.zeros(nt)
```

```
    for i in range(nt):
```

```
        fixed[i] = np.interp(R_FIXED, r_moving[i], C_tab[i])
```

```
        surf[i] = C_tab[i, -1]
```

```

t_end = s.t_end_s
# 表6: 每6h + 结束行; 列 0,0.5,1.0,表面
rows_t, rows_lbl = [], []
tv = 6 * 3600.0
while tv < t_end - 1:
    rows_t.append(tv)
    rows_lbl.append(f"{tv/3600:.0f}")
    tv += 6 * 3600.0
rows_t.append(t_end)
rows_lbl.append(f"{t_end/3600:.4f} (结束)")
table = []
for tv in rows_t:
    i = int(np.argmax(np.abs(s.tout - tv)))
    row = [fmt4(np.interp(rr, r_moving[i], s.C[i])) for rr in [0.0,
0.5, 1.0]]
    row.append(fmt4(s.C[i, -1]))
    table.append(row)
# 写 result4.xlsx
import openpyxl
wb = openpyxl.Workbook()
ws = wb.active
ws.title = "Sheet1"
ws.append(["时间\\到药材中心的距离"] + [fmt4(r) for r in R_FIXED] +
[R_SURF_NAME])
for i, tv in enumerate(s.tout):
    if tv <= 0 or tv > t_end + 1e-9:
        continue
    if abs(tv - round(tv / 60.0) * 60.0) < 1e-6 or i == nt - 1:
        ws.append([fmt4(tv)] + [fmt4(v) for v in fixed[i]] + [fmt4(surf[i])])
wb.save(U.os.path.join(U.RESULTS_DIR, "result4.xlsx"))
# 无收缩对照 (同物性附录4, R=2恒定)
s_ns = run(N=40, dt=1.0, shrink=False, threshold=U.THRESHOLD)
res = dict(s.stats)
res.update({
    "rows_h": rows_lbl,
    "cols_cm": [0.0, 0.5, 1.0, "药材表面"],
    "table_C": table,
    "t_end_s": fmt4(t_end), "t_end_h": fmt4(t_end / 3600.0),
    "t_end_noshrink_h": fmt4(s_ns.t_end_s / 3600.0),
    "R_final_cm": fmt4(s.R_cm[-1]),
    "mass_err": s.mass_err,
    "mass_err_noshrink": s_ns.mass_err,
})
save_json(res, "problem4.json")
np.savez_compressed(U.os.path.join(U.OUT_DIR, "problem4_fields.npz"),
    tout=s.tout, C=C_tab, fixed=fixed, surf=surf,
    r_moving=r_moving,

```

```

        R_cm=s.R_cm, t_end=t_end,
        tout_ns=s_ns.tout, C_ns=s_ns.C)
figures(s, s_ns, fixed, surf, r_moving)
print("P4 done %.1fs t_end=%.4f h; 无收缩对照 %.4f h" %
      (s.stats["runtime_s"], res["t_end_h"], res["t_end_noshrink_h"]))
print("表6:", rows_lbl, table)

def figures(s, s_ns, fixed, surf, r_moving):
    import matplotlib.pyplot as plt
    import plot_utils as PU
    PU.setup()
    tt_h = s.tout / 3600.0
    # 图1: 剖面族 (物理坐标) + 判停线
    fig, ax = plt.subplots(figsize=(6.0, 4.2))
    for hv in [6, 12, 18, 24, 30]:
        i = int(np.searchsorted(tt_h, hv))
        if i < len(tt_h) - 1:
            ax.plot(r_moving[i], s.C[i], label=f"{hv} h")
    ax.plot(r_moving[-1], s.C[-1], "k:", lw=1.4,
            label=f"{tt_h[-1]:.1f} h (结束)")
    ax.axhline(U.THRESHOLD, color="red", ls="--", lw=1.0, label="判停线 0.15")
    ax.set_xlabel("到中心距离 r / cm")
    ax.set_ylabel("C / (kg·kg$^{-1}$)")
    ax.legend(fontsize=8, ncols=2)
    fig.tight_layout()
    PU.save(fig, "fig_q4_profiles.pdf")
    # 图2: 收缩 vs 不收缩: 表面 C 与中心 C, 标注两模型干燥前沿
    fig, axes = plt.subplots(1, 2, figsize=(9.4, 3.8), sharey=True)
    axes[0].plot(tt_h, s.C[:, 0], label="中心 (收缩)")
    axes[0].plot(tt_h, s.C[:, -1], label="表面 (收缩)")
    tn_h = s_ns.tout / 3600.0
    axes[0].plot(tn_h, s_ns.C[:, 0], "--", color="gray", label="中心 (不收缩对照)")
    axes[0].axhline(0.15, color="red", lw=0.9, ls=":")
    axes[0].set_yscale("log")
    axes[0].set_xlabel("t / h"); axes[0].set_ylabel("C / (kg·kg$^{-1}$)")
    axes[0].legend(fontsize=8)
    axes[1].plot(tt_h, s.R_cm, color="tab:purple")
    axes[1].set_xlabel("t / h"); axes[1].set_ylabel("药材半径 R(t) / cm",
                                                    color="tab:purple")
    i_end = int(np.argmax(np.abs(s.tout - s.t_end_s)))
    axes[1].plot(s.tout[i_end]/3600.0, s.R_cm[i_end], "o")
    axes[1].axvline(s.t_end_s / 3600, color="green", lw=1.0)
    axes[1].axvline(s_ns.t_end_s / 3600, color="gray", lw=1.0, ls="--")
    axes[1].legend(["R(t)", "t_end(收缩)", "t_end(对照)"], fontsize=8)
    fig.tight_layout()

```

```

PU.save(fig, "fig_q4_shrink.pdf")
# 图3: C(r,t) 热力图 (含收缩包络)
fig, ax = plt.subplots(figsize=(6.4, 4.2))
im = ax.pcolormesh(tt_h, np.linspace(0, 1, s.C.shape[1]),
                   s.C.T, shading="auto", cmap="viridis")
ax.plot(tt_h, s.R_cm / 2.0, "w--", lw=1.2, label="表面 ( $\xi=1$ ) ")
ax.set_xlabel("时间 t / h"); ax.set_ylabel(" $\xi = r/R(t)$ ")
ax.legend(fontsize=8)
cb = fig.colorbar(im, ax=ax, fraction=0.046)
cb.set_label("C / (kg·kg$^{-1}$)")
fig.tight_layout()
PU.save(fig, "fig_q4_heatmap.pdf")

if __name__ == "__main__":
    main()

```

code/sensitivity.py

"""灵敏度分析与数值检验：网格/步长收敛、参数扰动、边界条件与模型变体。

产出 results/sensitivity.json 与图 fig_conv.pdf、fig_sens_tornado.pdf、fig_variants.pdf。

"""

```

import time

import numpy as np

import utils as U
from utils import LENGTH_CM, Props, R0_CM, fmt4, save_json, simulate

def run_p3(**kw):
    kw.setdefault("N", 20)
    kw.setdefault("dt", 1.0)
    kw.setdefault("n_iter", 2)
    kw.setdefault("out_every", 600.0)
    kw.setdefault("threshold", U.THRESHOLD)
    kw.setdefault("t_max", 259200.0)
    kw.setdefault("props", Props(mode="att3"))
    return simulate(**kw)

def run_p4(**kw):
    kw.setdefault("N", 20)
    kw.setdefault("dt", 2.0)
    kw.setdefault("n_iter", 2)
    kw.setdefault("out_every", 600.0)

```

```

kw.setdefault("threshold", U.THRESHOLD)
kw.setdefault("t_max", 259200.0)
kw.setdefault("props", Props(mode="att4"))
t2, R2 = U.load_att2()
kw.setdefault("R_of_t_cm", lambda t: float(np.interp(t, t2, R2)))
return simulate(**kw)

def main():
    res = {}
    # ----- 1. 收敛性: 问题1 (N 与 dt 同时减半) -----
    t0 = time.time()
    s1_20 = simulate(t_max=1800, dt=0.25, N=20, props=Props(mode="att2"),
out_every=100., n_iter=2)
    s1_40 = simulate(t_max=1800, dt=0.0625, N=40, props=Props(mode="att2"),
out_every=100., n_iter=2)
    s1_80 = simulate(t_max=1800, dt=0.015625, N=80, props=Props(mode="att2"),
out_every=100., n_iter=2)
    rA = np.linspace(0, 2, 21)
    rB = np.linspace(0, 2, 41)
    rC = np.linspace(0, 2, 81)
    d20_40T = d20_40C = d40_80 = 0.0
    for i in [1, 3, 6, 9, 12, 15, 18]:
        d20_40T = max(d20_40T, float(np.max(np.abs(s1_20.T[i] - np.interp(rA,
rB, s1_40.T[i])))))
        d20_40C = max(d20_40C, float(np.max(np.abs(s1_20.C[i] - np.interp(rA,
rB, s1_40.C[i])))))
        d40_80 = max(d40_80, float(np.max(np.abs(s1_40.T[i] - np.interp(rB,
rC, s1_80.T[i])))))
        d40_80 = max(d40_80, float(np.max(np.abs(s1_40.C[i] - np.interp(rB,
rC, s1_80.C[i])))))
    res["p1_conv"] = {
        "maxdiff_T_N20_N40": fmt4(d20_40T),
        "maxdiff_C_N20_N40": fmt4(d20_40C),
        "maxdiff_N40_N80": fmt4(d40_80),
        "mass_err_N20": float(f"{s1_20.mass_err:.2e}"),
        "runtime_s": round(time.time() - t0, 1),
    }
    print("P1 conv:", res["p1_conv"], flush=True)

    # ----- 2. 问题3 网格/步长与守恒 -----
    t0 = time.time()
    s3_base = run_p3()
    s3_dt2 = run_p3(dt=2.0)
    s3_N40 = run_p3(N=40, dt=0.5)
    base = s3_base.t_end_s
    res["p3_conv"] = {

```

```

    "t_end_base_h": fmt4(base / 3600),
    "t_end_dt2_h": fmt4(s3_dt2.t_end_s / 3600),
    "t_end_N40dt05_h": fmt4(s3_N40.t_end_s / 3600),
    "dt2_rel_dev_pct": fmt4(100 * abs(s3_dt2.t_end_s - base) / base),
    "N40_rel_dev_pct": fmt4(100 * abs(s3_N40.t_end_s - base) / base),
    "mass_err": float(f"{s3_base.mass_err:.2e}"),
    "runtime_s": round(time.time() - t0, 1),
}
print("P3 conv:", res["p3_conv"], flush=True)

# ----- 3. 参数扰动 (±20%, 单因素) 对 t_end3 -----
sweep = {}
for name, key in [("h", "h"), ("h_m", "hm"), ("k", "k_scale"),
                  ("rho_cp", "rcp_scale"), ("D0", "d_scale")]:
    def t_for(factor):
        kw = dict(props=Props(mode="att3"))
        if key == "h":
            kw["h"] = U.H_CONV * factor
        elif key == "hm":
            kw["hm"] = U.HM_CONV * factor
        else:
            kw["props"] = Props(mode="att3", **{key: factor})
        return run_p3(**kw).t_end_s
    lo = t_for(0.8)
    hi = t_for(1.2)
    sweep[name] = {"base_h": fmt4(base / 3600), "m20_h": fmt4(lo / 3600),
                  "p20_h": fmt4(hi / 3600),
                  "rel_change_pct": fmt4(max(abs(lo - base), abs(hi -
base)) / base * 100)}
    print("P3", name, sweep[name], flush=True)
res["p3_sweep"] = sweep

# ----- 4. 边界条件/模型变体 (问题3) -----
s3_last = run_p3(att1_mode="lastpoint")
s3_end = run_p3(area_end_factor=1 + R0_CM / LENGTH_CM)
s3_latent = run_p3(latent=True)
res["p3_variants"] = {
    "t_end_lastpoint_h": fmt4(s3_last.t_end_s / 3600),
    "t_end_endface_corr_h": fmt4(s3_end.t_end_s / 3600),
    "t_end_latent_h": fmt4(s3_latent.t_end_s / 3600),
    "rel_lastpoint_pct": fmt4(100 * (s3_last.t_end_s - base) / base),
    "rel_endface_pct": fmt4(100 * (s3_end.t_end_s - base) / base),
    "rel_latent_pct": fmt4(100 * (s3_latent.t_end_s - base) / base),
}
print("P3 variants:", res["p3_variants"], flush=True)

# ----- 5. 问题4: 网格一致性、收缩收益、灵敏度 -----

```

```

t0 = time.time()
s4_base = run_p4()
s4_N40 = run_p4(N=40, dt=1.0)
s4_fixed = run_p4(R_of_t_cm=lambda t: 2.0)
base4 = s4_base.t_end_s
p4 = {"N40_rel_dev_pct": fmt4(100 * abs(s4_N40.t_end_s - base4) / base4),
      "t_end_base_h": fmt4(base4 / 3600),
      "t_end_noshrink_h": fmt4(s4_fixed.t_end_s / 3600),
      "shrink_speedup_pct": fmt4(100 * (1 - base4 / s4_fixed.t_end_s)),
      "mass_err": float(f"{s4_base.mass_err:.2e}")}
for name, key in [("h", "h"), ("h_m", "hm"), ("D0", "d_scale"),
                  ("k", "k_scale"), ("rho_cp", "rcp_scale")]:
    def t4_for(factor):
        kw = {}
        if key == "h":
            kw["h"] = U.H_CONV * factor
        elif key == "hm":
            kw["hm"] = U.HM_CONV * factor
        else:
            kw["props"] = Props(mode="att4", **{key: factor})
        return run_p4(**kw).t_end_s
    lo = t4_for(0.8)
    hi = t4_for(1.2)
    p4[name] = {"base_h": fmt4(base4 / 3600), "m20_h": fmt4(lo / 3600),
               "p20_h": fmt4(hi / 3600),
               "rel_change_pct": fmt4(max(abs(lo - base4), abs(hi -
base4))) / base4 * 100)}
    print("P4", name, p4[name], flush=True)
p4["runtime_s"] = round(time.time() - t0, 1)
res["p4"] = p4

# ----- 6. 物性参数集一致性 (附录2 vs 附录3, 问题1 30min) -----
s30 = simulate(t_max=1800, dt=0.25, N=20, props=Props(mode="att3"),
out_every=100., n_iter=2)
res["p1_vs_p2_at1800"] = {
    "T_center_att2": fmt4(s1_20.T[-1, 0]), "T_center_att3": fmt4(s30.T[-1,
0]),
    "T_surf_att2": fmt4(s1_20.T[-1, -1]), "T_surf_att3": fmt4(s30.T[-1,
-1]),
    "C_center_att2": fmt4(s1_20.C[-1, 0]), "C_center_att3": fmt4(s30.C[-1,
0]),
    "C_surf_att2": fmt4(s1_20.C[-1, -1]), "C_surf_att3": fmt4(s30.C[-1,
-1]),
}
print("P1 vs P2 @1800:", res["p1_vs_p2_at1800"], flush=True)

save_json(res, "sensitivity.json")

```

```

figures(res)
print("SENS DONE", flush=True)

def figures(res):
    import matplotlib.pyplot as plt
    import plot_utils as PU
    PU.setup()
    fig, axes = plt.subplots(1, 2, figsize=(9.2, 3.6))
    axes[0].bar(["T (N20-N40)", "C (N20-N40)", "max (N40-N80)"],
                [res["p1_conv"]["maxdiff_T_N20_N40"],
                 res["p1_conv"]["maxdiff_C_N20_N40"],
                 res["p1_conv"]["maxdiff_N40_N80"]],
                width=0.45, color=["tab:blue", "tab:orange", "tab:green"])
    for k, v in zip(range(3), [res["p1_conv"]["maxdiff_T_N20_N40"],
                               res["p1_conv"]["maxdiff_C_N20_N40"],
                               res["p1_conv"]["maxdiff_N40_N80"]]):
        axes[0].text(k, v + 2e-4, f"{v:.4f}", ha="center", fontsize=8.5)
    axes[0].set_ylabel("表中值最大绝对偏差")
    axes[0].set_title("问题1 网格/步长收敛 (N、dt 减半)", fontsize=10)
    axes[1].bar(["dt=2 vs dt=1", "N40 vs N20"],
                [res["p3_conv"]["dt2_rel_dev_pct"], res["p3_conv"]
                 ["N40_rel_dev_pct"]],
                width=0.4, color="tab:purple")
    for k, v in zip(range(2), [res["p3_conv"]["dt2_rel_dev_pct"],
                               res["p3_conv"]["N40_rel_dev_pct"]]):
        axes[1].text(k, v + 0.02, f"{v:.3f}%", ha="center", fontsize=8.5)
    axes[1].set_ylabel("t_end 相对偏差 / %")
    axes[1].set_title("问题3 数值精度", fontsize=10)
    fig.tight_layout()
    PU.save(fig, "fig_conv.pdf")

# 龙卷风图 (问题3 与问题4 并排)
fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.8), sharey=False)
for ax, sw, ttl in [(axes[0], res["p3_sweep"], "问题3"),
                    (axes[1], {k: v for k, v in res["p4"].items() if
                               isinstance(v, dict)},
                      "问题4")]:
    names = list(sw.keys())
    lo = [sw[n]["base_h"] - sw[n]["m20_h"] for n in names]
    hi = [sw[n]["p20_h"] - sw[n]["base_h"] for n in names]
    order = np.argsort([max(abs(a), abs(b)) for a, b in zip(lo, hi)])
    names = [names[i] for i in order]
    lo = [lo[i] for i in order]
    hi = [hi[i] for i in order]
    y = np.arange(len(names))
    ax.barh(y - 0.18, lo, height=0.36, color="tab:blue", label="-20%")

```

```

    ax.barh(y + 0.18, hi, height=0.36, color="tab:red", label="+20%")
    ax.axvline(0, color="k", lw=0.8)
    ax.set_yticks(y, names)
    ax.set_xlabel(" $\Delta t_{end} / h$ ")
    ax.set_title(f"{ttl}:  $\pm 20\%$  参数扰动", fontsize=10)
    ax.legend(fontsize=8.5, loc="lower right")
fig.tight_layout()
PU.save(fig, "fig_sens_tornado.pdf")

# 变体对比柱
fig, ax = plt.subplots(figsize=(6.6, 3.6))
labels = ["基准\n(P3)", "末点恒温", "端面修正", "潜热耦合", "基准\n(P4)", "P4
不收缩"]
vals = [res["p3_conv"]["t_end_base_h"],
        res["p3_variants"]["t_end_lastpoint_h"],
        res["p3_variants"]["t_end_endface_corr_h"],
        res["p3_variants"]["t_end_latent_h"],
        res["p4"]["t_end_base_h"],
        res["p4"]["t_end_noshrink_h"]]
colors = ["tab:gray", "tab:gray", "tab:gray", "tab:gray", "tab:purple",
"tab:purple"]
bars = ax.bar(labels, vals, color=colors, width=0.55)
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width() / 2, v + 0.5, f"{v:.2f}",
ha="center", fontsize=8.5)
ax.set_ylabel(" $t_{end} / h$ ")
ax.set_title("模型变体下烘干时长对比", fontsize=10)
fig.tight_layout()
PU.save(fig, "fig_variants.pdf")

if __name__ == "__main__":
    main()

```